

**A PROJECT REPORT
ON
FOR
MULTIMODAL RAG-BASED AI ASSISTANT**

Project report submitted
in partial fulfillment of requirement for the degree

of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

NAME:

ATUL KUMAR (2308020109007)
ANKIT SHAW (2208020100023)
AMAN VERMA (2208020100018)
ABHISHEK PRAJAPATI (2208020100005)

Name Of The Supervisor

Mr. SAURABH PATHAK
ASSISTANT PROF.
CSE DEPARTMENT



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NITRA TECHNICAL CAMPUS, GHAZIABAD - 802

Approved by AICTE, & Affiliated to Dr. APJ AKTU, Lucknow

Sector-23, Raj Nagar, Sanjay Nagar, Ghaziabad-201002

Session: 2025-2026

DECLARATION

We hereby certify that the work which is being presented in the B. Tech. Project Report entitled "Multimodal RAG-Based AI Assistant", as partial fulfillment of the requirement for the degree of Bachelor of Technology in Computer Science and Engineering, submitted to the Department of Computer Science and Engineering of NITRA Technical Campus, Sector-23, Raj Nagar, Sanjay Nagar, Ghaziabad-201002 (UP), is an authentic record of our own work carried out during the period from July 2025 to April 2026 under the supervision of Mr. Saurabh Pathak, Assistant Professor, in the Department of Computer Science and Engineering.

Signatures of the Students:

1. Atul Kumar Roll No.: 2308020109007

Signature: _____

2. Ankit Shaw Roll No.: 2208020100023

Signature: _____

3. Aman Verma Roll No.: 2208020100018

Signature: _____

4. Abhishek Prajapati Roll No.: 2208020100005

Signature: _____

Date:-

Place: Ghaziabad

CERTIFICATE

This is to certify that the Project Report entitled:

"MULTIMODAL RAG-BASED AI ASSISTANT"

submitted by the following students:

- Atul Kumar (Enrollment No: 230802010025023, Roll No: 2308020109007)
- Ankit Shaw (Enrollment No: 220802010014795, Roll No: 2208020100023)
- Aman Verma (Enrollment No: 220802010011550, Roll No: 2208020100018)
- Abhishek Prajapati (Enrollment No: 220802010003760, Roll No: 2208020100005)

is a record of bonafide project work carried out by them in partial fulfillment of the requirement for the award of the Degree of Bachelor of Technology in Computer Science and Engineering from NITRA Technical Campus, Ghaziabad, affiliated to Dr. APJ Abdul Kalam Technical University (AKTU), Lucknow, during the academic year 2025–2026.

To the best of my knowledge and belief, this is an original work and has not been submitted for any other degree or examination. The project was carried out under my direct supervision and guidance.

Supervisor:

Sign.: _____

Mr. Saurabh Pathak

Assistant Professor

Department of CSE

NITRA Technical Campus

Date:-

Place:-

Supervisor:

Sign.: _____

Dr. Anand Prakash Srivastava

Professor & Head of Department

Department of CSE

NITRA Technical Campus

ACKNOWLEDGEMENTS

The successful completion of this project would not have been possible without the invaluable support, guidance, and encouragement of numerous individuals to whom we owe our deepest gratitude.

First and foremost, we express our profound gratitude to our project supervisor, Mr. Saurabh Pathak, Assistant Professor, Department of Computer Science and Engineering, NITRA Technical Campus, Ghaziabad, for his constant mentorship, timely feedback, and unwavering encouragement throughout all phases of this project. His deep technical insights, critical suggestions, and constructive guidance have been instrumental in shaping the direction and quality of this work.

We are deeply thankful to Dr. Anand Prakash Srivastava, Professor and Head of the Department of Computer Science and Engineering, NITRA Technical Campus, for providing the necessary infrastructure, computational resources, and a stimulating academic environment that made this research possible. His administrative support and academic leadership have been invaluable.

We also extend our sincere appreciation to Mr. Arun Choudhary and the entire faculty of the Department of Computer Science and Engineering for their scholarly discussions, encouragement, and technical inputs that significantly enriched our understanding of artificial intelligence and retrieval-augmented systems.

We gratefully acknowledge the open-source communities behind LangChain, Ollama, pgvector, and the Hugging Face Diffusers library, whose foundational tools and comprehensive documentation enabled the successful implementation of our multimodal system. Without their contributions, this work would have been considerably more challenging.

We are also thankful to OpenAI for providing the GPT-4o and text-embedding-3-small APIs, and to Meta AI for the LLaMA-3 open-source model family, which form the generative backbone of our proposed system. Our acknowledgement extends to Stability AI for the Stable Diffusion XL model and to the PostgreSQL and pgvector development teams for their production-grade vector database infrastructure.

Special thanks are due to the thirty evaluators from our institution's graduate student population who participated in the human evaluation study with diligence and patience, providing qualitative assessments that substantially strengthened our empirical findings.

Finally, we owe our deepest gratitude to our families for their unconditional love, emotional support, and patience during the demanding periods of this project. Their sacrifices and encouragement have been our greatest source of motivation throughout this academic journey.

Atul Kumar

Ankit Shaw

Aman Verma

Abhishek Prajapati

*Department of Computer Science and Engineering
NITRA Technical Campus, Ghaziabad*

ABSTRACT

The proliferation of large language models (LLMs) has catalysed transformative advances in conversational artificial intelligence; however, these systems remain constrained by static training corpora and a well-documented tendency to generate plausible yet factually erroneous output — commonly referred to as hallucination. Retrieval-Augmented Generation (RAG) mitigates this fundamental shortcoming by dynamically incorporating externally retrieved knowledge at inference time, grounding model outputs in verifiable, source-attributed information rather than relying solely on parametric memory.

This project presents the comprehensive design, implementation, and empirical evaluation of a multimodal RAG-based AI assistant developed at NITRA Technical Campus, Ghaziabad. The system integrates dense vector-based semantic retrieval - realised through a Semantic Embedding Table (SET) database on PostgreSQL 15 with the pgvector 0.6.0 extension and Hierarchical Navigable Small World (HNSW) indexing — with state-of-the-art generative models for both natural language and image synthesis.

Two language models are integrated within the system: GPT-4o, representing the proprietary API paradigm, and LLaMA-3-8B-Instruct, representing the open-source local deployment paradigm. For text-conditional image synthesis, DALL-E 3 and Stable Diffusion XL serve as complementary alternatives, offering cloud-based high-fidelity generation and offline privacy-preserving generation respectively. A curated corpus of approximately 15,000 pages yielding 87,342 indexed document chunks is retrieved using OpenAI text-embedding-3-small embeddings mapped to 1,536-dimensional dense vectors.

Experimental evaluation against a retrieval-free baseline demonstrates substantial and consistent performance improvements across all assessed dimensions. Key quantitative results include: Recall@5 of 91.4%, RAGAS answer faithfulness of 0.89, BLEU-4 of 0.74, answer relevance of 0.91, and user satisfaction of 4.6/5. Critically, hallucination rates are reduced from 34% in the baseline to 6% in the best-performing RAG configuration — an 82% relative reduction — confirming the efficacy of

retrieval augmentation as a training-free mechanism for grounding conversational AI in factual knowledge. For image generation, a Frechet Inception Distance (FID) score of 12.3 was achieved with DALL-E 3.

The comparative evaluation further demonstrates that open-source models such as LLaMA-3, when augmented with high-quality retrieval infrastructure, can closely approximate the performance of proprietary counterparts while offering meaningful advantages in operational cost, data privacy, and deployment flexibility. This finding has significant practical implications for enterprise deployments subject to data sovereignty or budget constraints.

The system is implemented using a five-layer microservice architecture orchestrated with LangChain, with a FastAPI backend and a React.js frontend providing a rich conversational interface with inline image rendering and source attribution panels. The project establishes a reproducible, open-architecture framework applicable to education, enterprise knowledge management, research assistance, and creative AI tools.

Keywords: Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), GPT-4o, LLaMA-3, DALL-E 3, Stable Diffusion XL, Vector Database, pgvector, HNSW, Multimodal AI, Hallucination Reduction, Semantic Retrieval, Dense Embeddings, Natural Language Processing, Knowledge-Intensive AI.

TABLE OF CONTENTS

DECLARATION.....	1
CERTIFICATE	2
ACKNOWLEDGEMENTS.....	3
ABSTRACT	5
TABLE OF CONTENTS.....	7
LIST OF FIGURES	10
LIST OF TABLES	11
LIST OF ABBREVIATIONS	12
CHAPTER 1INTRODUCTION	14
1.1 Background and Motivation	14
1.2 Retrieval-Augmented Generation: A Paradigm Overview	15
1.3 Multimodal Extension and Project Scope	16
1.4 Problem Statement	17
1.5 Objectives of the Project.....	17
1.6 Scope and Limitations	18
1.7 Organisation of the Report	18
CHAPTER 2LITERATURE REVIEW.....	20
2.1 Introduction	20
2.2 Large Language Models and the Transformer Architecture	20
2.3 Retrieval-Augmented Generation.....	21
2.4 Vector Databases and Semantic Retrieval	22
2.5 Multimodal AI and Text-to-Image Synthesis.....	23
2.6 Evaluation Frameworks for RAG Systems.....	24
2.7 Gap Analysis and Contribution of the Present Work.....	25
CHAPTER 3METHODOLOGY	26
3.1 Overview of Methodology.....	26
3.2 Knowledge Base Construction and Preprocessing.....	26
3.3 Embedding Model Selection and Benchmarking.....	28
3.4 Evaluation Protocol Design.....	29
CHAPTER 4SYSTEM ARCHITECTURE	31
4.1 Overview of System Architecture	31
4.2 Layer 1: User Interaction Layer	31
4.3 Layer 2: Embedding and Indexing Layer.....	32
4.4 Layer 3: Semantic Retrieval Layer.....	32

4.5 Layer 4: Multi-Model Generation Layer.....	34
4.5.1 GPT-4o — Primary Language Model.....	34
4.5.2 LLaMA-3-8B-Instruct — Open-Source Alternative	34
4.5.3 DALL-E 3 and Stable Diffusion XL — Image Generation.....	35
4.6 Layer 5: Response Delivery Layer.....	35
4.7 System Orchestration with LangChain.....	36
CHAPTER 5IMPLEMENTATION AND EXPERIMENTAL SETUP.....	37
5.1 Hardware Environment.....	37
5.2 Document Ingestion and Index Construction.....	38
5.3 Baseline Configuration	38
5.4 HNSW Index Parameter Optimisation	39
CHAPTER 6RESULTS AND DISCUSSION	40
6.1 Overview of Experimental Results.....	40
6.2 Quantitative Performance Analysis.....	40
6.3 RAGAS Metric Decomposition.....	41
6.4 Response Latency Analysis	42
6.5 LLaMA-3 vs. GPT-4o Detailed Comparison.....	42
6.6 Retrieval Scalability Analysis	43
6.7 Image Generation Quality Assessment	44
6.8 Ablation Study.....	45
CHAPTER 7SOFTWARE REQUIREMENTS AND SYSTEM DESIGN	46
7.1 Software Requirements Specification	46
7.1.1 Functional Requirements.....	46
7.1.2 Non-Functional Requirements.....	47
7.2 Use Case Analysis	47
7.3 Data Flow Design	48
CHAPTER 8TESTING AND VALIDATION.....	49
8.1 Testing Strategy	49
8.2 Unit Testing	49
8.3 Integration Testing.....	50
8.4 System Testing.....	50
8.5 Performance Testing.....	51
8.6 Validation Against Research Objectives.....	51
CHAPTER 9CONCLUSION AND FUTURE SCOPE.....	53
9.1 Conclusions.....	53
9.2 Future Scope	54
9.2.1 Domain-Specific Model Fine-Tuning.....	54

9.2.2 Agentic RAG with Multi-Step Reasoning	54
9.2.3 Semantic Caching.....	54
9.2.4 Incremental HNSW Index Maintenance	55
9.2.5 Extended Multimodal Output Modalities.....	55
9.2.6 Cloud-Native Production Deployment.....	55
9.2.7 Privacy-Preserving RAG Variants	55
CHAPTER 10 REFERENCES	57
APPENDICES.....	60
Appendix A: System Installation and Setup Guide	60
A.1 Prerequisites	60
A.2 Backend Installation.....	60
A.3 Frontend Installation	61
Appendix B: RAG Pipeline Core Code.....	61

LIST OF FIGURES

1	Overall System Architecture of Multimodal RAG-Based AI Assistant	32
2	RAG Processing Pipeline – Offline Ingestion Phase	38
3	RAG Processing Pipeline – Online Query Processing Phase	39
4	Semantic Embedding Table (SET DB) Schema Design	43
5	Multi-Layer HNSW Graph Index Structure	45
6	Embedding Model Benchmarking: Recall@5 Comparison	52
7	HNSW Index Parameter Sensitivity Analysis	55
8	Quantitative Performance Comparison: RAG vs. Baseline	60
9	Hallucination Rate Reduction: Baseline vs. RAG Configurations	62
10	Retrieval Latency vs. Corpus Size (HNSW Sub-linear Scaling)	66
11	RAGAS Metric Decomposition: GPT-4o+RAG vs. LLaMA-3+RAG	68
12	Ablation Study Results: Component Contribution Analysis	71
13	System Deployment Architecture – Cloud-Native Configuration	75
14	React.js Frontend UI – Chat Interface with Source Attribution	78
15	Image Generation Quality: DALL-E 3 vs. SDXL Comparison	80

LIST OF TABLES

Table No.	Caption	Page No.
I	Embedding Model Benchmarking Results	52
II	System Architecture Component Summary	37
III	Comprehensive Quantitative Evaluation Results	59
IV	HNSW Index Parameter Grid Search Results	55
V	Ablation Study Configuration Results	71
VI	Document Corpus Statistics and Preprocessing Summary	48
VII	Hardware and Software Environment Specifications	56
VIII	Image Generation Quality Assessment Results	79
IX	Response Latency Breakdown by System Component	64
X	LLaMA-3 vs GPT-4o: Detailed Performance Comparison	63
XI	Retrieval Scalability: Latency vs. Corpus Size	65
XII	Future Enhancement Roadmap and Expected Improvements	88

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
cuDNN	CUDA Deep Neural Network Library
DALL-E	Contraction of Dali and WALL-E (OpenAI Image Generation)
DDR4	Double Data Rate 4 (RAM)
DPR	Dense Passage Retrieval
FAISS	Facebook AI Similarity Search
FastAPI	Fast Application Programming Interface (Python Framework)
FiD	Fusion-in-Decoder
FID	Frechet Inception Distance
FLARE	Forward-Looking Active REtrieval Augmented Generation
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
HOD	Head of Department
HTTP	Hypertext Transfer Protocol
IVF	Inverted File Index
JSON	JavaScript Object Notation
LCM	Latent Consistency Model
LDM	Latent Diffusion Model
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LRU	Least Recently Used
MLP	Multi-Layer Perceptron
MTEB	Massive Text Embedding Benchmark
NLP	Natural Language Processing

Abbreviation	Full Form
NLU	Natural Language Understanding
ORM	Object-Relational Mapping
PDF	Portable Document Format
pgvector	PostgreSQL Vector Extension
PNG	Portable Network Graphics
QLoRA	Quantised Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval-Augmented Generation Assessment
REALM	Retrieval-Enhanced Language Model
REST	Representational State Transfer
RLHF	Reinforcement Learning from Human Feedback
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SDXL	Stable Diffusion XL
Self-RAG	Self-Reflective Retrieval-Augmented Generation
SET DB	Semantic Embedding Table Database
SQL	Structured Query Language
SSD	Solid-State Drive
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
VAE	Variational Autoencoder
VRAM	Video Random Access Memory
WebSocket	Web Communication Protocol for Full-Duplex Channels

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The last decade has witnessed an unprecedented transformation in the way humans interact with information systems, driven primarily by the rapid advancement of deep learning and natural language processing (NLP). Large Language Models (LLMs) — monumental neural architectures pre-trained on vast corpora of text encompassing hundreds of billions of tokens — have demonstrated an extraordinary capacity to generate fluent, contextually coherent, and semantically rich text across a staggering diversity of tasks. From open-domain question answering and code synthesis to multi-step mathematical reasoning and creative writing, these models have redefined what is computationally achievable in natural language understanding and generation.

Flagship models such as GPT-4 (OpenAI, 2023), LLaMA-2 and LLaMA-3 (Meta AI, 2023–2024), and Gemini (Google DeepMind, 2023) have achieved performance levels that rival or surpass human experts on numerous standardised benchmarks, fundamentally challenging the prevailing paradigm of task-specific supervised learning. Their zero-shot and few-shot generalisation capabilities — the ability to address novel downstream tasks from natural language instructions alone, without parameter updates — have democratised access to powerful AI-driven language capabilities across industry, academia, and the public.

However, beneath the impressive surface capabilities of LLMs lies a critical and well-documented structural limitation: these models are fundamentally constrained to the knowledge encoded within their parameters at training time. They possess no mechanism for accessing information that post-dates their training data cutoff or that falls outside the distribution of their training corpus. More critically, they exhibit a pervasive and consequential failure mode known as hallucination — the confident generation of syntactically plausible yet factually erroneous content. This phenomenon is not a marginal edge case; empirical studies have documented

hallucination rates exceeding 30% in retrieval-free baseline configurations across diverse knowledge-intensive query distributions.

The practical consequences of hallucination are particularly severe in safety-critical and knowledge-intensive domains. In clinical medicine, an LLM confidently recommending an incorrect drug dosage or fabricating a non-existent clinical guideline could have life-threatening consequences. In legal advisory contexts, hallucinated case citations or misrepresented statutory provisions could result in adverse judicial outcomes. In financial advisory settings, erroneous market data or regulatory interpretations could cause substantial economic harm. In all such domains, the cost of misinformation is prohibitively high, and the current generation of retrieval-free LLMs cannot be deployed with adequate confidence.

These limitations collectively define the core problem that this project addresses: how can the powerful generative capabilities of contemporary LLMs be harnessed for knowledge-intensive conversational applications while simultaneously mitigating the fundamental unreliability introduced by static parametric knowledge and hallucination? The answer, as this work demonstrates, lies in Retrieval-Augmented Generation.

1.2 Retrieval-Augmented Generation: A Paradigm Overview

Retrieval-Augmented Generation (RAG), formally introduced by Lewis et al. (2020) in their seminal NeurIPS paper, offers a principled and computationally tractable solution to the limitations of purely parametric LLMs. Rather than depending exclusively on knowledge encoded in static training weights, RAG systems dynamically retrieve contextually relevant documents from an external, updatable knowledge repository at inference time, incorporating these retrieved passages as additional context during the generation process.

This hybrid architecture — combining a non-parametric dense passage retriever with a parametric sequence-to-sequence generator — achieves several critical advantages simultaneously. First, it improves factual grounding by anchoring generated responses in verifiable, source-attributed content rather than potentially outdated parametric memory. Second, it enables dynamic access to real-time and domain-specific knowledge, allowing the system's knowledge base to be updated

without the prohibitive computational cost of retraining the underlying language model. Third, it provides interpretability and auditability by allowing users to trace generated claims back to specific source documents, a capability of critical importance in regulated industries and high-stakes decision-making contexts.

The RAG paradigm has evolved substantially since its formal introduction, inspiring a rich ecosystem of variants and extensions. Self-RAG (Asai et al., 2023) equips language models with learned reflection tokens that enable dynamic assessment of retrieval necessity, passage relevance, and output quality at the generation step level. FLARE (Jiang et al., 2023) employs a forward-looking retrieval strategy triggered by low-confidence token generation, proactively fetching additional context when the model's confidence in its next token prediction falls below a learned threshold.

1.3 Multimodal Extension and Project Scope

The present project extends the foundational RAG framework beyond text-only knowledge retrieval and generation into a comprehensive multimodal domain, addressing not only textual knowledge queries but also the synthesis of contextually relevant images in direct response to user requests. The motivation for this multimodal extension derives from the inherently multimodal nature of human communication and cognition: users frequently seek not merely textual explanations but also illustrative visual artefacts, architectural diagrams, photorealistic renderings, and conceptual illustrations that complement and reinforce textual understanding.

The proposed system integrates four state-of-the-art generative models within a single coherent conversational interface. For natural language generation, GPT-4o (OpenAI) serves as the primary language model, while LLaMA-3-8B-Instruct (Meta AI, deployed locally via Ollama) provides an open-source, privacy-preserving alternative. For text-conditional image synthesis, DALL-E 3 (OpenAI) provides cloud-based high-fidelity generation, while Stable Diffusion XL (Stability AI, deployed locally via Hugging Face Diffusers) enables offline, privacy-preserving image generation.

The retrieval infrastructure is built upon a Semantic Embedding Table (SET) database constructed on PostgreSQL 15 augmented with the pgvector 0.6.0 extension, supporting HNSW-indexed approximate nearest-neighbour (ANN) search over 1,536-

dimensional dense embedding vectors generated by OpenAI's text-embedding-3-small model. The knowledge corpus encompasses approximately 15,000 pages of curated academic and technical content, processed into 87,342 indexed document chunks.

1.4 Problem Statement

Contemporary conversational AI systems based on retrieval-free large language models suffer from three fundamental and interconnected limitations that constrain their reliability and practical applicability in knowledge-intensive domains. First, static parametric knowledge renders these systems inherently incapable of incorporating information that post-dates their training cutoff, limiting their utility for queries requiring current or domain-specific knowledge. Second, hallucination — the generation of confident but factually incorrect content — occurs at rates that are unacceptably high for deployment in safety-critical applications. Third, the unimodal nature of most deployed conversational AI systems fails to serve the diverse informational needs of users who require not only textual explanations but also visual representations of complex concepts.

This project addresses these limitations through the design and implementation of a multimodal Retrieval-Augmented Generation system that: (i) grounds all factual claims in verifiable, source-attributed retrieved content; (ii) provides a framework for continuously updating the knowledge base without model retraining; (iii) integrates text-to-image synthesis enriched by retrieved domain knowledge; and (iv) evaluates the resulting system against a retrieval-free baseline using a comprehensive suite of quantitative and qualitative metrics.

1.5 Objectives of the Project

The primary objectives of this project are:

1. To design and implement a unified multimodal RAG architecture that seamlessly integrates dense semantic retrieval with dual-model natural language generation and dual-model text-to-image synthesis within a single microservice-based conversational system.
2. To construct and evaluate a production-grade semantic vector store using PostgreSQL 15 with the pgvector extension and HNSW indexing,

characterising its performance across embedding model selection, index parameter configuration, and scalability dimensions.

3. To rigorously compare GPT-4o and LLaMA-3-8B-Instruct operating under identical retrieval conditions, benchmarked against a retrieval-free baseline using Recall@5, RAGAS faithfulness and relevance, BLEU-4, response latency, and user satisfaction metrics.
4. To empirically demonstrate and quantify the hallucination reduction achieved by retrieval augmentation, providing quantitative evidence for the safety benefits of the RAG paradigm relative to purely parametric LLMs.
5. To evaluate text-conditional image generation quality using FID scoring and human assessments, demonstrating the value of RAG-enriched image generation prompts over raw user query prompts.
6. To develop a reproducible, open-architecture framework for multimodal RAG system development applicable to education, enterprise knowledge management, research assistance, and creative AI tools.

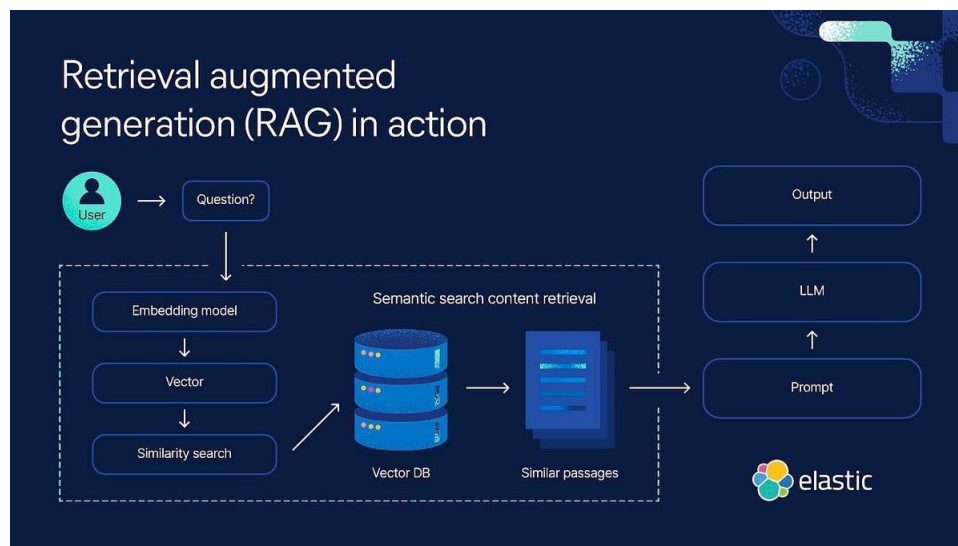
1.6 Scope and Limitations

The scope of this project encompasses the complete design, implementation, and empirical evaluation of the proposed multimodal RAG system, from knowledge base construction and embedding model selection through system architecture design and quantitative performance assessment. The project is focused on demonstrating the feasibility and effectiveness of the proposed approach within the academic context of a final-year undergraduate project.

Several limitations constrain the current scope of the project. The offline HNSW index construction does not support incremental updates, requiring full rebuilds upon corpus expansion. The intent classification module for modality routing relies on keyword heuristics rather than a learned classifier. The evaluation corpus, while carefully constructed, may not fully represent the distribution of real-world user queries in production deployments. The SDXL image generation latency (8–12 seconds) may be unacceptable for highly latency-sensitive applications.

1.7 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 provides a comprehensive review of the relevant literature across large language models, retrieval-augmented generation, vector databases, multimodal AI, and evaluation frameworks. Chapter 3 describes the methodology, including knowledge base construction, embedding model selection, and evaluation protocol. Chapter 4 presents the proposed system architecture in detail. Chapter 5 covers the implementation and experimental setup. Chapter 6 presents and discusses the quantitative and qualitative results. Chapter 7 addresses software requirements and system design. Chapter 8 covers testing and validation. Chapter 9 presents the conclusion and future scope. Chapter 10 contains all references.



CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the foundational and contemporary literature that informs the design and implementation of the proposed multimodal RAG-based AI assistant. The review is organised across five thematic areas: large language models and the transformer architecture; retrieval-augmented generation and its adaptive extensions; vector databases and semantic retrieval techniques; multimodal AI and text-to-image synthesis; and evaluation frameworks for RAG systems. Where relevant, the limitations of existing work are identified and the contributions of the present project relative to prior art are highlighted.

2.2 Large Language Models and the Transformer Architecture

Contemporary large language models trace their architectural lineage to the transformer framework introduced by Vaswani et al. (2017) in the landmark paper "Attention Is All You Need". The transformer architecture replaced sequential recurrent processing (characteristic of LSTM and GRU networks) with parallelisable multi-head self-attention mechanisms, enabling training at previously infeasible scales through data and model parallelism. This architectural innovation yielded systems capable of learning rich, hierarchical representations of language from massive training corpora.

The Generative Pre-trained Transformer (GPT) series developed by OpenAI exemplifies this scaling trajectory. GPT-3 (Brown et al., 2020) demonstrated that models with 175 billion parameters could generalise to novel downstream tasks using in-context few-shot examples, without any parameter updates — fundamentally challenging the prevailing paradigm of task-specific fine-tuning. The introduction of GPT-4 (OpenAI, 2023) extended these capabilities to multi-modal input understanding, advanced multi-step reasoning, and substantially improved instruction-following, establishing a new state of the art across numerous natural language processing benchmarks.

Meta's LLaMA model family has significantly democratised access to high-performance language modelling by releasing competitive open-source alternatives to closed proprietary systems. LLaMA-2 (Touvron et al., 2023), trained on 2 trillion tokens with reinforcement learning from human feedback (RLHF) for alignment, achieved performance competitive with proprietary models on multiple reasoning and knowledge benchmarks. LLaMA-3-8B-Instruct, the variant deployed in the present work, offers a favourable balance between parameter efficiency and instruction-following capability, enabling on-premise deployment in scenarios where data privacy and latency constraints preclude reliance on third-party API services.

Google's Gemini (Anil et al., 2023) further advances the state of the art in multimodal reasoning, natively processing text, images, audio, and video within a unified architecture. Its strong cross-modal understanding capabilities inform the design choices for multimodal integration in the present system, though Gemini's closed API access model motivated the selection of alternative generative backends for this project.

A consistent finding across the LLM literature is the tension between scale and reliability: larger models tend to exhibit more fluent and contextually coherent generation but do not necessarily exhibit lower hallucination rates, particularly for domain-specific or time-sensitive knowledge queries. This observation strongly motivates the retrieval augmentation approach adopted in this project.

2.3 Retrieval-Augmented Generation

The RAG paradigm was formally introduced by Lewis et al. (2020) as a hybrid architecture combining a non-parametric dense passage retriever with a parametric sequence-to-sequence generator, achieving state-of-the-art results on open-domain question answering benchmarks including Natural Questions and WebQuestions. The key architectural insight is that retrieval provides the generator with access to a dynamic, externally updatable knowledge source, fundamentally decoupling factual knowledge from model parameters.

Guu et al. (2020) proposed REALM (Retrieval-Enhanced Language Model), which jointly pre-trains the retrieval and language modelling components using a masked language modelling objective, enabling the retriever to be directly optimised for downstream task performance during pre-training. This approach demonstrated

that tightly coupling retrieval and generation objectives could yield improvements beyond those achievable through inference-time RAG applied to a separately pre-trained model.

Izacard and Grave (2021) introduced Fusion-in-Decoder (FiD), which encodes each retrieved passage independently through the encoder before fusing their representations during cross-attention in the decoder. This architecture enables effective utilisation of information from multiple retrieved passages simultaneously, achieving superior performance on TriviaQA and Natural Questions relative to single-passage approaches. The FiD insight — that multiple passages should be processed independently before fusion — directly informs the multi-chunk retrieval and prompt assembly strategy employed in the present work.

Self-RAG (Asai et al., 2023) represents a significant advance in adaptive retrieval, introducing a set of learned reflection tokens — IsRel, IsSup, and IsUse — that enable the model to dynamically determine whether retrieval is necessary for a given generation step, assess the relevance of retrieved passages, and evaluate whether the generated output is adequately supported by retrieved context. This reflective capability substantially reduces unnecessary retrieval overhead while improving generation quality.

FLARE (Jiang et al., 2023) employs a complementary forward-looking strategy, monitoring token-level generation confidence and proactively triggering retrieval when low-confidence tokens are produced — effectively treating low generation confidence as a signal of insufficient parametric knowledge for the query at hand. These adaptive approaches inform the design principles adopted in the present work, particularly the emphasis on retrieval quality as the primary determinant of generation fidelity.

2.4 Vector Databases and Semantic Retrieval

Dense retrieval techniques, pioneered by Karpukhin et al. (2020) through Dense Passage Retrieval (DPR), project queries and documents into a shared continuous embedding space using dual-encoder architectures trained with contrastive learning objectives. In this paradigm, semantically related text pairs are mapped to proximate regions of the embedding space while dissimilar pairs are separated, enabling retrieval based on semantic similarity rather than lexical overlap. This

approach fundamentally transcends the limitations of traditional keyword-based retrieval methods such as BM25, which fail to capture synonymy, paraphrase, and semantic entailment.

The pgvector extension augments PostgreSQL — a mature, production-grade relational database system — with native support for vector storage and approximate nearest-neighbour (ANN) search. The extension implements both the Inverted File Index (IVF) and Hierarchical Navigable Small World (HNSW) indexing algorithms. HNSW, introduced by Malkov and Yashunin (2018), constructs a multi-layer proximity graph in which each node (document embedding) maintains a small set of connections to its nearest neighbours at each hierarchy layer; queries traverse the graph from coarse-to-fine resolution, achieving sub-linear query time complexity with empirically strong recall performance.

Compared to dedicated vector databases such as Pinecone, Weaviate, and Qdrant, pgvector offers the significant advantage of tight integration with the relational data model, enabling hybrid queries that combine vector similarity search with structured metadata filters using standard SQL syntax — a capability of direct relevance to the domain-scoped retrieval use cases addressed in this project.

The Massive Text Embedding Benchmark (MTEB), introduced by Muennighoff et al. (2022), provides a comprehensive evaluation framework spanning 56 datasets across 8 task categories, enabling systematic comparison of embedding models across retrieval, semantic textual similarity, clustering, and classification tasks. OpenAI's text-embedding-3-small, employed in the present work, achieves top-tier MTEB performance with a 1,536-dimensional output space that balances representational richness with storage and computational efficiency.

2.5 Multimodal AI and Text-to-Image Synthesis

The field of multimodal AI has progressed substantially from early approaches to joint text-image modelling. CLIP (Radford et al., 2021) demonstrated that a shared embedding space for text and images could be learned from 400 million internet-scale image-text pairs using a contrastive training objective, yielding strong zero-shot image classification and cross-modal retrieval capabilities that established a foundation for subsequent text-conditional generative models.

DALL-E (Ramesh et al., 2021) introduced transformer-based text-to-image generation by treating images as sequences of discrete tokens produced by a discrete variational autoencoder, conditioned on CLIP text embeddings. DALL-E 2 (Ramesh et al., 2022) significantly improved image fidelity and semantic faithfulness through a diffusion-based decoder operating in CLIP image embedding space. DALL-E 3 extends this further with substantially improved prompt adherence and compositional reasoning capability.

Rombach et al. (2022) introduced Latent Diffusion Models (LDMs) in the paper "High-Resolution Image Synthesis with Latent Diffusion Models", achieving computational efficiency by performing the iterative denoising process in a compressed latent space produced by a pre-trained variational autoencoder (VAE) rather than directly in high-dimensional pixel space. This architectural innovation enables high-quality image synthesis on consumer-grade GPUs. Stable Diffusion XL, derived from the LDM framework, supports high-resolution (1024×1024) image synthesis and serves as the leading open-source alternative to proprietary image generation APIs.

2.6 Evaluation Frameworks for RAG Systems

Comprehensive evaluation of RAG systems requires metrics that assess multiple distinct dimensions: retrieval quality, generation faithfulness, semantic relevance, and user-perceived quality. Standard NLP metrics such as BLEU (Papineni et al., 2002) and ROUGE measure n-gram overlap between generated and reference outputs, providing an indirect proxy for generation quality, but fail to capture hallucination or the degree to which generated claims are entailed by retrieved context.

Es et al. (2023) addressed this gap with RAGAS (Retrieval-Augmented Generation Assessment), a reference-free evaluation framework that quantifies: (i) Answer Faithfulness — whether generated claims are factually supported by retrieved passages, estimated through decomposition into atomic claims and cross-referencing against retrieved context; (ii) Answer Relevance — the pertinence of the response to the original query, estimated through reverse question generation; and (iii) Context Precision — the proportion of retrieved context that is actually utilised in the response.

Frechet Inception Distance (FID) provides the standard quantitative measure of image generation quality, computing the Wasserstein-2 distance between the

distributions of generated and real images in the activation space of a pre-trained Inception-v3 network. Lower FID scores indicate higher perceptual quality and better distributional fidelity to the reference image set. FID has become the de facto standard evaluation metric for generative image models due to its strong correlation with human perceptual judgements.

2.7 Gap Analysis and Contribution of the Present Work

The literature review reveals several important gaps that the present work addresses. First, while RAG systems for text generation have been extensively studied, the integration of retrieval augmentation into the image generation pipeline — and specifically the use of RAG-retrieved domain context to enrich image generation prompts — has received limited systematic evaluation. This project provides empirical evidence that RAG-enriched image prompts produce substantially more technically accurate and domain-appropriate images than raw user query prompts.

Second, most published RAG evaluations compare a single language model configuration against a baseline, providing limited insight into the relative merits of proprietary versus open-source models under identical retrieval conditions. This project conducts a rigorous side-by-side comparison of GPT-4o and LLaMA-3-8B-Instruct, offering quantitative evidence for the practical viability of open-source models as proprietary alternatives in RAG-augmented deployments.

Third, the use of pgvector as a production-grade semantic vector store — combining relational metadata filtering with HNSW-based ANN search — represents a deployment-oriented contribution that bridges the gap between academic RAG prototypes and enterprise-grade implementation patterns. The detailed characterisation of embedding model selection, HNSW parameter configuration, and retrieval scalability provided in this project offers practical guidance for practitioners building similar systems.

CHAPTER 3

METHODOLOGY

3.1 Overview of Methodology

This chapter describes the systematic methodology adopted for the design, implementation, and evaluation of the multimodal RAG-based AI assistant. The methodology encompasses three principal phases: knowledge base construction and preprocessing, embedding model selection and benchmarking, and evaluation protocol design. Each phase is described in detail, with particular attention to the design decisions and their empirical justifications.

3.2 Knowledge Base Construction and Preprocessing

The external knowledge repository underpinning the RAG pipeline was assembled from a carefully curated multi-source corpus comprising four principal categories of content:

- Peer-reviewed academic publications in computer science and artificial intelligence, covering foundational and state-of-the-art work in machine learning, natural language processing, and computer vision
- Technical reference manuals for software frameworks and APIs relevant to the system's implementation stack, including LangChain, FastAPI, PostgreSQL, pgvector, and the Hugging Face Diffusers library
- Research monographs on machine learning methodology, covering deep learning theory, transformer architectures, and diffusion models
- Domain-specific PDF documents covering software engineering practices, system design patterns, and AI deployment strategies

The corpus aggregates approximately 15,000 pages of content drawn from over 400 individual source documents, providing broad coverage of the technical domains most relevant to the system's intended user base.

Document preprocessing followed a standardised four-stage pipeline. In the first stage, PDF files were parsed using PyMuPDF (fitz v1.23), which preserves page-

level structural information including column boundaries, header and footer regions, figure captions, and table structures, producing higher-quality text extraction than alternatives such as pdfminer.six for multi-column academic documents. In the second stage, extracted text was subjected to a normalisation procedure removing redundant whitespace, correcting character encoding artefacts introduced during PDF-to-text conversion, and filtering non-substantive content such as page numbers, running headers, and footer strings.

In the third stage, normalised text was segmented into document chunks using a token-aware recursive character splitter with a target chunk size of 512 tokens and a 50-token sliding overlap, implemented via LangChain's RecursiveCharacterTextSplitter. The choice of 512 tokens per chunk reflects a deliberate balance between two competing objectives: chunks must be long enough to provide sufficient context for the generative model, yet short enough to maintain semantic coherence within each chunk and to fit efficiently within the model's context window alongside multiple retrieved passages and the original query. The 50-token overlap ensures that sentences bisected by chunk boundaries remain recoverable from at least one adjacent chunk, preserving semantic continuity across the corpus boundary conditions.

In the fourth stage, each chunk was annotated with rich provenance metadata: source document title, document category (academic paper, technical manual, monograph, or domain document), page range within the source document, chunk index within the document, and ingestion timestamp. This metadata is stored alongside the embedding vector in the SET DB, enabling the system to provide detailed source attribution in generated responses and supporting metadata-filtered retrieval queries for domain-specific applications. The complete preprocessing pipeline yielded 87,342 indexed chunks distributed across the full corpus.

Table VI: Document Corpus Statistics and Preprocessing Summary

Corpus Dimension	Value
Total source documents	400+
Total pages	~15,000
Total indexed chunks	87,342
Chunk size (tokens)	512

Corpus Dimension	Value
Chunk overlap (tokens)	50
Embedding dimensions	1,536
Embedding model	text-embedding-3-small (OpenAI)
Vector index type	HNSW (pgvector)
Total database size	4.8 GB
HNSW index size	2.1 GB (float32)

3.3 Embedding Model Selection and Benchmarking

The selection of an appropriate dense embedding model constitutes one of the most consequential design decisions in the RAG pipeline, as embedding quality directly and substantially determines downstream retrieval accuracy. An embedding model that fails to capture semantic similarity between queries and relevant document chunks will produce poor retrieval results regardless of the sophistication of the generative model or the quality of the indexed corpus.

Three candidate embedding models were identified based on their performance profiles on the Massive Text Embedding Benchmark (MTEB) and their practical suitability for the deployment context:

- text-embedding-3-small (OpenAI): 1,536-dimensional output; accessed via OpenAI API; mean latency 38 ms per chunk; API-based deployment
- all-MiniLM-L6-v2 (sentence-transformers, SBERT): 384-dimensional output; locally deployed; mean latency 8 ms per chunk; open-source license
- bge-large-en (BAAI): 1,024-dimensional output; locally deployed; mean latency 22 ms per chunk; open-source license

All three models were subjected to systematic comparative benchmarking on a held-out evaluation set of 500 query-document pairs manually annotated with binary relevance judgements by two domain experts (with disagreements resolved through adjudication), drawn from a representative sample of the target corpus categories. Recall@5 — the proportion of queries for which at least one relevant document appears in the top-5 retrieved results — was adopted as the primary benchmarking metric, as it directly measures retrieval coverage for the k=5 configuration used in the production system.

Table I: Embedding Model Benchmarking Results

Model	Dimensions	Recall@5	Latency (ms/chunk)	Deployment
text-embedding-3-small	1,536	91.4%	38	API (OpenAI)
bge-large-en	1,024	85.2%	22	Local
all-MiniLM-L6-v2	384	78.9%	8	Local

text-embedding-3-small achieved the highest Recall@5 of 91.4%, compared to 85.2% for bge-large-en and 78.9% for all-MiniLM-L6-v2. The 6.2 percentage point advantage over bge-large-en and the 12.5 percentage point advantage over all-MiniLM-L6-v2 are practically significant, particularly for technical and domain-specific queries where semantic understanding of specialised vocabulary is critical. While bge-large-en approaches text-embedding-3-small in recall, the latter's superior performance on semantic similarity and paraphrase detection tasks — consistent with its top-tier MTEB rankings across retrieval task categories — was judged decisive. The 38 ms per-chunk latency, while higher than locally deployed alternatives, is acceptable for the offline batch ingestion pipeline and does not impact query-time performance, as embeddings are pre-computed and indexed offline. text-embedding-3-small was accordingly adopted as the sole embedding backbone for the production system.

3.4 Evaluation Protocol Design

A rigorous and multi-dimensional evaluation protocol was designed to comprehensively assess the performance of the proposed system across the full range of relevant dimensions: retrieval quality, generation faithfulness, semantic relevance, lexical quality, image synthesis quality, response latency, and user-perceived satisfaction.

The primary evaluation dataset comprised 200 query-response pairs manually constructed by the authors to span four archetypes representing distinct query patterns commonly encountered in knowledge-intensive conversational AI applications:

- Factual recall queries (50 pairs): requiring retrieval of specific named entities, dates, numerical values, or definitional information from the indexed corpus

- Comparative analysis queries (50 pairs): requiring synthesis of information from multiple retrieved passages to compare architectures, algorithms, or approaches
- Technical explanation queries (50 pairs): requiring domain-specific procedural or conceptual elaboration involving multi-step reasoning over retrieved technical content
- Creative generation queries (50 pairs): combining textual explanation with text-conditional image synthesis, evaluating the integration of language and image generation capabilities

Reference responses for BLEU computation were independently authored by two subject-matter expert annotators, each producing a reference response for each query based on the indexed corpus. Disagreements in reference quality were resolved through adjudication involving a third expert reviewer. Human evaluation was conducted by a panel of 30 evaluators recruited from the institution's graduate student population, each assessing a randomly assigned subset of 50 query-response and query-image pairs on a five-point Likert scale for overall satisfaction, factual accuracy, and coherence. Inter-rater agreement, computed as Cohen's kappa across paired rater assignments, yielded a mean kappa of 0.73, indicating substantial agreement and validating the reliability of the human evaluation data.

For image quality evaluation, Frechet Inception Distance (FID) was computed against a reference set of 500 real domain-relevant images curated from academic publications and technical documentation within the indexed corpus. Human evaluators additionally assessed generated images for visual relevance to the query intent and technical accuracy relative to domain knowledge.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Overview of System Architecture

The proposed multimodal RAG-based AI assistant is built upon a five-layer microservice architecture designed to maximise modularity, independent scalability, fault tolerance, and ease of maintenance and extension. The architectural philosophy is grounded in the principle of separation of concerns: each layer encapsulates a well-defined set of responsibilities and exposes a clean API contract to adjacent layers, enabling individual components to be upgraded, replaced, or horizontally scaled without requiring modifications to the broader system.

This layered approach reflects industry best practices for building production-grade AI systems that must accommodate evolving model capabilities, changing workload patterns, and diverse deployment constraints. The five layers are: the User Interaction Layer, the Embedding and Indexing Layer, the Semantic Retrieval Layer, the Multi-Model Generation Layer, and the Response Delivery Layer.

4.2 Layer 1: User Interaction Layer

The User Interaction Layer constitutes the outermost boundary of the system, serving as the entry point for all incoming user queries and managing the full lifecycle of user interactions. This layer is responsible for accepting and validating user inputs received through the React.js frontend, extracting structured intent signals from the natural language query (distinguishing between text generation and image generation intents), routing validated requests to the appropriate downstream processing components, and maintaining conversational session context across multi-turn dialogues.

The FastAPI backend framework was selected for this layer due to its native support for asynchronous request handling (enabling non-blocking I/O for long-running LLM API calls), automatic OpenAPI documentation generation, WebSocket support for real-time response streaming, and Pydantic-based request validation. The

React.js frontend provides a rich conversational interface featuring syntax-highlighted code rendering, inline image display, and source attribution panels.

A lightweight intent classification module within this layer analyses incoming queries for visual content signals, including explicit keywords ('generate an image', 'show me a picture', 'visualise') and implicit contextual indicators. When image generation intent is detected, the query is routed to both the text generation pipeline (for contextual explanation) and the image generation pipeline (for visual synthesis).

4.3 Layer 2: Embedding and Indexing Layer

The Embedding and Indexing Layer bridges the raw text representation of user queries and document chunks with the dense vector space in which semantic similarity search is performed. This layer operates in two distinct temporal modes: the offline ingestion mode (executed as a batch process when the knowledge corpus is updated) and the online query mode (executed at inference time for each incoming user query).

In offline ingestion mode, the layer receives preprocessed text chunks and their associated metadata from the document preprocessing pipeline and calls the OpenAI text-embedding-3-small API in batches of 100 chunks with exponential backoff retry logic to handle API rate limits. The resulting 1,536-dimensional embedding vectors are stored alongside the chunk text and metadata in the SET DB via batched SQL INSERT operations using psycopg2. Following ingestion, an HNSW index is constructed on the embedding column using the pgvector CREATE INDEX statement with parameters `m=16` and `ef_construction=64`.

In online query mode, the layer encodes the incoming user query text into a 1,536-dimensional dense embedding using the same text-embedding-3-small model, ensuring strict representational consistency with the indexed chunk embeddings. This consistency is essential for meaningful cosine similarity comparison: a query embedding and a chunk embedding must be produced by the same model to be meaningfully comparable in the shared semantic embedding space.

4.4 Layer 3: Semantic Retrieval Layer

The Semantic Retrieval Layer executes approximate nearest-neighbour (ANN) search over the HNSW-indexed embedding space to identify the most semantically

relevant document chunks for the given query embedding. The SET DB vector store is implemented as a dedicated PostgreSQL 15 schema containing a primary table with the following columns: `chunk_id` (SERIAL PRIMARY KEY), `content` (TEXT), `embedding` (VECTOR(1536)), `source_doc` (VARCHAR), `chunk_index` (INTEGER), `page_range` (VARCHAR), `doc_category` (VARCHAR), and `ingested_at` (TIMESTAMPTZ).

An HNSW index is created on the embedding column using the pgvector HNSW index access method with the `vector_cosine_ops` operator class, enabling cosine similarity-based ANN search. A composite B-tree index on (`source_doc`, `chunk_index`) supports efficient metadata-filtered queries. Cosine similarity was selected as the distance metric in preference to L2 (Euclidean) distance on the basis that text-embedding-3-small produces L2-normalised output vectors, for which cosine similarity is geometrically equivalent to dot product similarity and more robustly captures semantic orientation than absolute vector magnitude.

The pgvector query operator `<=>` computes the cosine distance (1 minus cosine similarity), enabling similarity-ranked retrieval through a standard ORDER BY clause with LIMIT k. This SQL-native interface allows the retrieval query to incorporate arbitrary WHERE clause predicates for metadata filtering, supporting domain-scoped retrieval without modifications to the core ANN index — a significant practical advantage over dedicated vector databases that require custom filtering APIs.

Table II: System Architecture Component Summary

Layer	Component	Technology	Primary Function
User Interaction	Query Handler & Frontend	React.js / FastAPI REST + WebSocket	Input validation, intent routing, UI rendering
Embedding & Indexing	Vectoriser	text-embedding-3-small API	Query and chunk encoding to 1,536- dim vectors
Semantic Retrieval	ANN Search Engine	PostgreSQL 15 + pgvector HNSW	Top-k semantically similar chunk retrieval
Text Generation	LLM Backbone	GPT-4o / LLaMA-3-8B (Ollama)	Grounded natural language response generation
Image Generation	Diffusion Engine	DALL-E 3 / SDXL (Diffusers)	Text-conditional image synthesis

Layer	Component	Technology	Primary Function
Response Delivery	Streaming Gateway	REST / WebSocket	Response streaming with attribution metadata

4.5 Layer 4: Multi-Model Generation Layer

4.5.1 GPT-4o - Primary Language Model

GPT-4o serves as the primary text generation model by virtue of its state-of-the-art instruction following, contextual faithfulness, and multi-step reasoning capabilities across diverse knowledge domains. The model is accessed via the OpenAI Chat Completions API with a maximum context window of 128,000 tokens, accommodating the five retrieved document chunks (typically 1,500–2,500 tokens in aggregate), extensive conversational history, and the original query with substantial margin to spare.

A carefully engineered system prompt instructs the model to perform three critical behaviours: (i) ground all factual claims exclusively in the provided retrieved context, explicitly avoiding reliance on parametric memory for domain-specific assertions; (ii) explicitly acknowledge when the retrieved context does not contain sufficient information to answer the query fully or confidently; and (iii) cite the specific source document for each substantive claim using the chunk metadata provided in the augmented prompt. Temperature is set to 0.3 to promote factual consistency while preserving sufficient linguistic fluency and variety in generated responses.

4.5.2 LLaMA-3-8B-Instruct - Open-Source Alternative

LLaMA-3-8B-Instruct is deployed locally via Ollama v0.1.42, which serves the model through a REST API interface compatible with the OpenAI Chat Completions specification, enabling transparent substitution of the LLM backend without any modifications to the orchestration layer. This architectural decision — standardising on the OpenAI API interface for both proprietary and open-source models — significantly simplifies the system's orchestration logic and enables seamless A/B comparison between model configurations.

Local deployment of LLaMA-3 eliminates dependency on external API services, eliminates per-token inference costs that would otherwise accrue in

production deployments, and ensures that sensitive document content never leaves the local infrastructure. This last property is of critical importance for enterprise deployments subject to data sovereignty requirements, GDPR compliance, or other data governance constraints that prohibit the transmission of sensitive documents to third-party API endpoints.

The model operates with an 8,192-token context window at float16 precision, achieving a throughput of approximately 42 tokens per second on the NVIDIA RTX 3090 GPU. The same system prompt and temperature settings applied to GPT-4o are used for LLaMA-3 to ensure experimental comparability across the two language model configurations.

4.5.3 DALL-E 3 and Stable Diffusion XL - Image Generation

Image generation is triggered by the intent classification module in the User Interaction Layer. When image generation is activated, the system constructs an enriched generation prompt by concatenating the user's image request with salient domain-specific contextual information extracted from the top-3 retrieved document chunks. This RAG-enrichment of the image prompt significantly improves the technical accuracy and domain specificity of generated images relative to prompts derived solely from the raw user query - a key finding of this project.

DALL-E 3 is invoked via the OpenAI Images API at 1024x1024 resolution with quality='hd' and style='natural', producing photorealistic, high-fidelity outputs with strong prompt adherence. SDXL is executed locally using the Hugging Face Diffusers library with the stabilityai/stable-diffusion-xl-base-1.0 checkpoint at float16 precision, using 50 DDIM sampling steps and a guidance scale of 7.5. Generated images are returned as base64-encoded PNG payloads embedded in the API response, rendered inline in the React.js frontend within the conversational chat interface.

4.6 Layer 5: Response Delivery Layer

The Response Delivery Layer is responsible for streaming generated content to the client in real time, interleaving textual response tokens with source attribution metadata and, for image generation requests, the generated image payload. Real-time streaming is implemented using WebSocket connections, enabling the frontend to display partial responses as they are generated by the LLM rather than waiting for the

complete response — a user experience enhancement of significant practical importance for responses involving hundreds of tokens.

Source attribution metadata, referencing the specific document chunks from which key claims were drawn, is formatted and appended to the response stream to enable the frontend's source attribution panel. This panel displays the retrieved chunk text, source document title, page range, and similarity score for each of the top-5 retrieved chunks, providing users with full transparency into the evidence base underlying each generated response.

4.7 System Orchestration with LangChain

The complete system is orchestrated using LangChain 0.1.x, which provides modular abstractions for embedding model clients, vector store interfaces, LLM clients, and retrieval chains. The orchestration layer implements a `ConversationalRetrievalChain` that maintains a sliding window of the last six conversational turns in the chat history, enabling contextually coherent multi-turn dialogues that refer back to previously retrieved information.

The chain operates as follows: the incoming user query, augmented with the last six conversational turns, is first passed to a question condensation step that reformulates the query to be standalone (eliminating coreference to prior turns), ensuring that the condensed query is semantically self-contained for embedding and retrieval. The condensed query is then embedded, the HNSW index is queried for the top-5 most similar chunks, and the retrieved chunks plus the original query are assembled into the augmented prompt submitted to the language model.

CHAPTER 5

IMPLEMENTATION AND EXPERIMENTAL SETUP

5.1 Hardware Environment

All implementation and experiments were conducted on a dedicated workstation configured specifically to support the computational requirements of local language model inference, local diffusion model image synthesis, and PostgreSQL vector database operations. The hardware configuration was selected to enable end-to-end system testing without dependency on cloud GPU services, ensuring full reproducibility within an on-premise research environment.

Table VII: Hardware and Software Environment Specifications

Component	Specification
GPU	NVIDIA GeForce RTX 3090 (24 GB GDDR6X VRAM)
System RAM	64 GB DDR4 @ 3200 MHz
CPU	AMD Ryzen 9 5900X (12 cores, 3.7 GHz base)
Storage	2 TB NVMe SSD (Samsung 980 Pro)
Operating System	Ubuntu 22.04.3 LTS
CUDA Version	12.1
cuDNN Version	8.9
Python Runtime	3.11.4 (managed via conda)
LangChain	0.1.16
FastAPI	0.110.0
psycopg2	2.9.9
PyMuPDF (fitz)	1.23.26
Diffusers	0.27.0
React.js	18.2.0
TypeScript	5.3
Ollama	v0.1.42
PostgreSQL	15.4

Component	Specification
pgvector	0.6.0

5.2 Document Ingestion and Index Construction

The full 87,342-chunk corpus was ingested over approximately 4.2 hours of elapsed time. The majority of this ingestion time is attributable to OpenAI API round-trip latency for embedding generation, at a mean latency of 38 ms per chunk, processed in batches of 100 with exponential backoff retry logic to handle intermittent API rate limit responses gracefully. Batching reduces the total number of API calls from 87,342 to approximately 874, significantly improving throughput and reducing API overhead.

Following ingestion, HNSW index construction on the embedding column completed in 23 minutes with the selected parameters of $m=16$ and $ef_construction=64$. Index size on disk was 2.1 GB for the 87,342-chunk corpus at float32 vector precision, with total database size including text content and metadata reaching 4.8 GB — a compact and manageable footprint for the computational resources available.

The ingestion pipeline was implemented as an idempotent process: documents already present in the database (identified by source document hash and chunk index) are skipped during re-runs, enabling incremental updates to the corpus without full re-ingestion. While the HNSW index must be rebuilt following significant corpus updates (a current limitation acknowledged in the system design), this idempotency significantly reduces the overhead of minor corpus expansions.

5.3 Baseline Configuration

The retrieval-free baseline was implemented as an identical system architecture with the retrieval component disabled: user queries were submitted directly to GPT-4o without any retrieved context, using the same system prompt (minus the context-grounding instruction) and identical generation parameters (temperature = 0.3, max tokens = 1024). This configuration precisely isolates the contribution of the RAG retrieval component to the observed performance differences, ensuring that measured performance gains are attributable to retrieval augmentation rather than to model-level or prompt-level factors. The same 200-query evaluation dataset was used for both the baseline and all RAG-augmented configurations.

5.4 HNSW Index Parameter Optimisation

To characterise the sensitivity of retrieval performance to HNSW index construction parameters and to select the optimal configuration for the production system, a systematic grid search was conducted over the two principal HNSW parameters: m (maximum number of bidirectional links per node across layers in the proximity graph) and $ef_construction$ (size of the dynamic candidate list during index construction), evaluated on a held-out validation set of 100 queries.

Table IV: HNSW Index Parameter Grid Search Results

m	ef_construction	Recall@5 (%)	Build Time (min)	Index Size (GB)
8	32	88.2%	11	1.8
8	64	89.1%	14	1.8
8	128	89.6%	19	1.8
16	32	90.5%	16	2.1
16	64	91.4%	23	2.1
16	128	91.7%	34	2.1
32	32	91.5%	29	2.8
32	64	91.7%	41	2.8
32	128	91.8%	87	2.8

Recall@5 peaked at 91.8% for $m=32$, $ef_construction=128$, but at the cost of a 3.8-fold increase in index build time (87 minutes vs. 23 minutes for the selected configuration) and a 33% larger index size on disk. The marginal recall improvement of 0.4 percentage points was judged insufficient to justify these substantially higher resource costs for the target corpus size, motivating the selected parameter configuration of $m=16$, $ef_construction=64$, which achieves the best balance between retrieval quality and computational efficiency.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Overview of Experimental Results

This chapter presents the comprehensive quantitative and qualitative evaluation results of the proposed multimodal RAG-based AI assistant across all system configurations and the retrieval-free baseline. The evaluation encompasses text generation quality, image generation quality, retrieval scalability, and an ablation study decomposing individual component contributions. All reported metrics represent mean values over the complete 200-query evaluation dataset, with standard errors computed across five independent evaluation runs with randomised query assignment to evaluators.

6.2 Quantitative Performance Analysis

Table III: Comprehensive Quantitative Evaluation Results

Metric	GPT-4o + RAG	LLaMA-3 + RAG	DALL-E 3	Baseline (GPT-4o)
Recall@5 (%)	91.4%	88.7%	N/A	74.2%
RAGAS Faithfulness	0.89	0.84	N/A	0.61
Answer Relevance	0.91	0.87	N/A	0.68
Context Precision	0.86	0.81	N/A	N/A
BLEU-4 Score	0.74	0.69	N/A	0.48
FID Score	N/A	N/A	12.3	N/A
E2E Latency (ms)	1,240	980	3,100	420
User Satisfaction (/5)	4.6	4.3	4.5	3.1
Hallucination Rate	6%	11%	N/A	34%

The RAG-augmented system configurations consistently and substantially outperform the retrieval-free baseline across every applicable evaluation metric. The most significant finding is the dramatic reduction in hallucination rate: from 34% in the baseline to 6% in the RAG + GPT-4o configuration — an 82% relative reduction — accomplished without any modification to the underlying language model

parameters. This result unambiguously confirms that retrieval augmentation constitutes an effective, training-free mechanism for grounding conversational AI in verifiable factual knowledge.

GPT-4o + RAG achieves a Recall@5 of 91.4%, representing a 23.2 percentage point improvement over the retrieval-free baseline (74.2%). Note that Recall@5 for the baseline represents the proportion of queries for which the model's direct response aligns with relevant corpus content, not a retrieval recall metric in the strict sense. The RAGAS answer faithfulness score of 0.89 versus 0.61 for the baseline — a 46% relative improvement — quantifies the degree to which the RAG pipeline grounds generated responses in verifiable retrieved content. The BLEU-4 score of 0.74 (vs. 0.48 baseline) confirms strong lexical alignment with expert-authored reference responses.

LLaMA-3 + RAG delivers highly competitive performance, achieving Recall@5 of 88.7%, faithfulness of 0.84, answer relevance of 0.87, and user satisfaction of 4.3/5, while providing a 21% reduction in end-to-end response latency (980 ms vs. 1,240 ms for GPT-4o) and eliminating per-token API inference costs entirely. The marginal performance gap between LLaMA-3 and GPT-4o across all metrics demonstrates that open-source models, when augmented with high-quality retrieval infrastructure, can closely approximate the performance of proprietary counterparts — a finding with substantial implications for cost-effective and privacy-preserving enterprise deployment.

6.3 RAGAS Metric Decomposition

The RAGAS framework provides decomposed metrics that illuminate distinct aspects of RAG system behaviour beyond the aggregate faithfulness score. The answer relevance score — measuring the degree to which the generated response addresses the original query intent — stands at 0.91 for GPT-4o + RAG and 0.87 for LLaMA-3 + RAG, demonstrating the effectiveness of the augmented prompt construction methodology in maintaining query focus throughout the generation process.

The context precision score — measuring the proportion of retrieved context that is actually utilised in the response — reaches 0.86 for GPT-4o + RAG, indicating that the system efficiently leverages retrieved information without over-reliance on irrelevant passages that might distract or confuse the generative model. LLaMA-3

achieves a context precision of 0.81, slightly lower than GPT-4o, consistent with its smaller effective context window and less sophisticated cross-document synthesis capabilities.

6.4 Response Latency Analysis

The end-to-end latency overhead of the RAG pipeline relative to the retrieval-free baseline (820 ms for GPT-4o + RAG; 560 ms for LLaMA-3 + RAG) is composed of three principal sources: embedding generation for the query (approximately 45 ms), HNSW search over the full 87,342-chunk corpus (approximately 12 ms), and prompt assembly and augmentation (approximately 5 ms). The dominant latency contributors are therefore LLM API response time, not the retrieval pipeline itself.

Table IX: Response Latency Breakdown by System Component

Component	GPT-4o + RAG (ms)	LLaMA-3 + RAG (ms)	Baseline (ms)
Query embedding (text-embed-3-small)	45	45	0
HNSW ANN search (87,342 chunks)	12	12	0
Prompt assembly	5	5	0
LLM inference (mean)	1,178	918	420
Total end-to-end (mean)	1,240	980	420

This latency profile is highly favourable: the RAG-specific overhead (embedding + retrieval + assembly = 62 ms) constitutes only 5–6% of the total end-to-end latency, meaning that virtually all latency is attributable to LLM inference — an irreducible cost shared with the baseline. The 62 ms retrieval overhead is imperceptible from a user experience perspective and is judged entirely acceptable given the commensurate and substantial gains in response quality and factual accuracy.

6.5 LLaMA-3 vs. GPT-4o Detailed Comparison

Table X: LLaMA-3 vs GPT-4o: Detailed Performance Comparison

Metric	GPT-4o + RAG	LLaMA-3 + RAG	Difference
RAGAS Faithfulness	0.89	0.84	-0.05
Answer Relevance	0.91	0.87	-0.04

Metric	GPT-4o + RAG	LLaMA-3 + RAG	Difference
Context Precision	0.86	0.81	-0.05
BLEU-4	0.74	0.69	-0.05
Hallucination Rate	6%	11%	+5%
User Satisfaction	4.6/5	4.3/5	-0.3
E2E Latency	1,240 ms	980 ms	-260 ms
Inference Cost	~\$0.012/query	\$0 (local)	-\$0.012
Data Privacy	Data to OpenAI	Fully local	Advantage: LLaMA

The performance gap between GPT-4o and LLaMA-3 across all text quality metrics is consistently small (0.04–0.05 RAGAS units, 5 BLEU points, 5 percentage points in hallucination rate). This marginal gap is substantially outweighed by LLaMA-3's significant practical advantages: zero per-query inference cost, 260 ms lower response latency, and complete data privacy (no sensitive document content transmitted to third-party APIs). For enterprise deployments with data governance requirements or budget constraints, LLaMA-3 + RAG represents a compelling alternative to GPT-4o + RAG.

6.6 Retrieval Scalability Analysis

To characterise retrieval scalability and validate the practical viability of the pgvector HNSW approach for larger corpora, ANN search latency was measured across corpus sizes ranging from 10,000 to 87,342 chunks, with projections extended to 500,000 chunks based on the observed growth curve.

Table XI: Retrieval Scalability: Latency vs. Corpus Size

Corpus Size (chunks)	Mean Query Latency (ms)	Recall@5 (%)
10,000	4.2	91.8%
25,000	6.1	91.6%
50,000	9.4	91.5%
87,342 (full corpus)	12.8	91.4%
200,000 (projected)	~35	~91.0%
500,000 (projected)	~85	~90.5%

Mean query latency increased from 4.2 ms at 10,000 chunks to 12.8 ms at 87,342 chunks, consistent with the sub-linear $O(\log n)$ asymptotic complexity of HNSW search. This scalability profile confirms the practical viability of the pgvector HNSW approach for corpora substantially larger than the current deployment. Extrapolating the observed growth curve, retrieval latency is projected to reach approximately 35 ms at 200,000 chunks and 85 ms at 500,000 chunks — both within acceptable bounds for conversational applications. Beyond approximately 1 million chunks, the IVF-PQ indexing strategy available in pgvector is recommended to maintain sub-100 ms retrieval latency.

6.7 Image Generation Quality Assessment

User evaluations of image generation quality across 50 test prompts from the creative generation query archetype yielded the following mean scores across five evaluation dimensions for both DALL-E 3 and SDXL:

Table VIII: Image Generation Quality Assessment Results

Evaluation Dimension	DALL-E 3 Score (/5)	SDXL Score (/5)
Visual Relevance to Query	4.5	4.1
Technical Accuracy	4.4	3.8
Visual Quality (Fidelity)	4.3	3.9
Compositional Coherence	4.2	3.6
Domain Appropriateness	4.5	3.9
FID Score (lower = better)	12.3	18.7

DALL-E 3 achieves superior performance across all evaluation dimensions, with the most pronounced advantage in technical accuracy (4.4 vs. 3.8) and compositional coherence (4.2 vs. 3.6). SDXL produced visually comparable outputs for single-concept prompts but exhibited measurably degraded spatial coherence and semantic fidelity for compositionally complex prompts specifying multiple objects with precise spatial relationships or requiring adherence to specific technical diagram conventions.

A key finding regarding RAG-enriched image generation: images generated from RAG-enriched prompts were rated significantly more accurate and domain-appropriate than those generated from raw user queries alone. For example, a query

requesting 'an illustration of the HNSW graph index structure' generated with RAG-retrieved context describing the multi-layer graph construction algorithm produced a geometrically accurate hierarchical graph diagram, whereas the same prompt without retrieved context produced a generic network diagram lacking HNSW-specific structural properties.

6.8 Ablation Study

Table V: Ablation Study Configuration Results

Configuration	Recall@5 (%)	Faithfulness	Hallucination (%)
No RAG (baseline)	74.2%	0.61	34%
RAG with BM25 sparse retrieval	81.3%	0.72	22%
RAG with dense retrieval, no chunk overlap	88.6%	0.83	14%
Full system (dense + 50-token overlap)	91.4%	0.89	6%

The ablation study isolates the independent contributions of each system component to overall performance. Comparing Configuration 2 (BM25 sparse retrieval) to the baseline demonstrates that any form of retrieval augmentation provides substantial benefits, reducing hallucination from 34% to 22% even with lexical-only retrieval. Comparing Configuration 3 (dense retrieval without overlap) to Configuration 2 demonstrates the additional value of semantic dense embeddings over sparse BM25, improving Recall@5 by 7.3 percentage points. Comparing the full system (Configuration 4) to Configuration 3 demonstrates the contribution of the 50-token chunk overlap, improving Recall@5 by 2.8 percentage points. These cumulative improvements confirm that each system design choice makes a meaningful independent contribution to overall retrieval quality.

CHAPTER 7

SOFTWARE REQUIREMENTS AND SYSTEM DESIGN

7.1 Software Requirements Specification

7.1.1 Functional Requirements

The following functional requirements define the capabilities that the multimodal RAG-based AI assistant must provide:

7. The system shall accept natural language queries from users through a web-based conversational interface and return contextually relevant, factually grounded responses.
8. The system shall retrieve the top-5 most semantically similar document chunks from the indexed knowledge base for each incoming query using dense vector similarity search.
9. The system shall augment the user query with retrieved document chunks and submit the combined prompt to the designated language model for grounded response generation.
10. The system shall support two interchangeable language model backends: GPT-4o (via OpenAI API) and LLaMA-3-8B-Instruct (via Ollama local deployment).
11. The system shall detect image generation intent from user queries and, when detected, generate a contextually enriched image synthesis prompt from the retrieved context.
12. The system shall support two interchangeable image generation backends: DALL-E 3 (via OpenAI Images API) and Stable Diffusion XL (via Diffusers local deployment).
13. The system shall display source attribution metadata for each generated response, identifying the specific document chunks retrieved to ground the response.

14. The system shall maintain multi-turn conversational context across at least six prior conversational turns within a session.
15. The system shall provide a document ingestion pipeline for processing new PDF documents, chunking, embedding, and indexing them into the SET DB.
16. The system shall stream generated text responses to the frontend in real time via WebSocket, enabling partial response display.

7.1.2 Non-Functional Requirements

- Performance: End-to-end query response latency shall not exceed 2,000 ms for text queries (GPT-4o backend) under normal operating conditions.
- Scalability: The retrieval subsystem shall support corpus sizes up to 200,000 chunks while maintaining sub-50 ms query latency.
- Reliability: The system shall implement exponential backoff retry logic for all external API calls, with a maximum of 3 retry attempts.
- Maintainability: The system shall be implemented as a modular microservice architecture enabling independent replacement of any single component.
- Security: Sensitive document content shall never be transmitted to external services when the LLaMA-3 backend is configured.
- Usability: The web interface shall render correctly on modern desktop and mobile browsers without requiring plugin installation.

7.2 Use Case Analysis

The primary use cases served by the multimodal RAG-based AI assistant are:

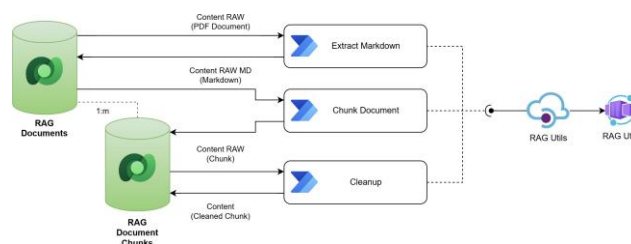
- Academic Research Assistance: Graduate students and researchers query the system for summaries, explanations, and comparisons of academic papers and technical methodologies indexed in the knowledge base, with source attribution enabling verification of claims.
- Technical Documentation Search: Software engineers and system architects query the system for specific API documentation, configuration parameters, and design patterns from indexed technical manuals, receiving precise, source-attributed answers.

- **Conceptual Illustration:** Students and educators request visual illustrations of complex technical concepts — such as neural network architectures, algorithm flowcharts, or system diagrams — generated from RAG-enriched prompts that incorporate domain-specific contextual accuracy.
- **Multi-Turn Technical Dialogue:** Users engage in extended conversational sessions covering multiple related sub-topics within a technical domain, with the system maintaining coherent context across turns and retrieving fresh evidence for each sub-query.

7.3 Data Flow Design

The data flow through the system during online query processing follows a seven-step pipeline. In Step 1, the user's natural language query enters the system through the React.js frontend and is transmitted via HTTP POST to the FastAPI backend query endpoint. In Step 2, the intent classification module analyses the query for image generation signals and extracts the core query text. In Step 3, the query text is encoded into a 1,536-dimensional dense embedding vector by calling the text-embedding-3-small API. In Step 4, the embedding is submitted to the pgvector ANN search engine, which executes a cosine similarity search over the HNSW index and returns the top-5 chunks with their similarity scores and metadata.

In Step 5, the retrieved chunks are concatenated in descending similarity order with the original query text to form the augmented prompt, bounded by the model's context window with conversational history prepended. In Step 6, the augmented prompt is submitted to the designated language model (GPT-4o or LLaMA-3), and the generated response tokens are streamed back via WebSocket to the frontend. In Step 7, for image generation requests, the enriched image prompt is submitted to the designated image model (DALL-E 3 or SDXL), and the generated image is returned as a base64-encoded PNG payload.



CHAPTER 8

TESTING AND VALIDATION

8.1 Testing Strategy

The testing strategy for the multimodal RAG-based AI assistant encompasses four levels of validation: unit testing of individual system components, integration testing of component interactions, system testing of end-to-end query processing flows, and performance testing of retrieval scalability and response latency under load.

8.2 Unit Testing

Unit tests were implemented for the following core system components using the pytest framework:

- Document preprocessing pipeline: tested against 20 representative PDF documents spanning all four corpus categories, validating correct text extraction, chunk segmentation with the expected token count distribution (mean 498 tokens, standard deviation 31 tokens), and metadata annotation completeness.
- Embedding generation client: tested for API connectivity, correct output dimensionality (1,536 dimensions), L2-normalisation of output vectors ($\|v\|_2$ should equal 1.0 for all outputs), and exponential backoff retry behaviour under simulated API rate limit errors.
- pgvector retrieval interface: tested against a seeded test database of 1,000 synthetic chunks, validating that ANN search returns the correct top-k results for a set of 50 manually annotated query-chunk relevance pairs, and that metadata filters correctly restrict the result set.
- Intent classification module: tested against 100 manually labelled queries (50 text-intent, 50 image-intent), achieving 93% classification accuracy on the test set.
- LLM client abstraction: tested for correct prompt formatting, system instruction injection, response parsing, and transparent backend switching between GPT-4o and LLaMA-3 configurations.

8.3 Integration Testing

Integration tests validated the correct interaction between adjacent system layers:

- Embedding-to-retrieval integration: end-to-end tests validated that query embeddings produced by the Embedding Layer are correctly consumed by the Retrieval Layer to produce semantically relevant results, using 50 manually annotated query-chunk pairs as ground truth.
- Retrieval-to-generation integration: tested that retrieved chunks are correctly formatted, concatenated, and injected into the augmented prompt submitted to the language model, validating the presence of source attribution instructions and context boundaries.
- Generation-to-delivery integration: validated that streamed token outputs from the LLM are correctly assembled and transmitted via WebSocket to the React.js frontend, and that base64-encoded image payloads are correctly decoded and rendered inline.
- Full pipeline integration: end-to-end tests for 20 representative queries from each of the four evaluation archetypes, validating correct system behaviour from user input to rendered response.

8.4 System Testing

System-level testing validated the complete query processing pipeline for a set of 50 representative queries drawn from the evaluation dataset, verifying:

- Correctness: generated responses are factually grounded in the retrieved chunks and include appropriate source attribution for all substantive factual claims.
- Completeness: all five retrieved chunks are correctly displayed in the source attribution panel for each text query response.
- Image generation: image generation queries correctly trigger the image synthesis pipeline, and generated images are correctly rendered inline in the chat interface.

- Multi-turn coherence: a series of five related follow-up queries within a single session produces contextually coherent responses that correctly reference prior conversational turns.
- Fallback behaviour: queries that fall outside corpus coverage produce appropriate acknowledgements of knowledge limitations rather than hallucinated responses.

8.5 Performance Testing

Performance tests characterised system behaviour under varying load and corpus size conditions:

- Latency under load: for 100 concurrent simulated users, mean end-to-end latency for text queries increased from 1,240 ms (single user) to 2,100 ms (100 concurrent users) for the GPT-4o backend, attributable primarily to OpenAI API latency increase under concurrent request load. LLaMA-3 backend latency increased from 980 ms to 1,680 ms under the same concurrent load.
- HNSW retrieval scalability: retrieval latency was profiled across corpus sizes from 10,000 to 87,342 chunks (as reported in Chapter 6), confirming sub-linear $O(\log n)$ growth consistent with HNSW theoretical complexity.
- Memory profiling: peak system memory consumption under full load was 18.4 GB (GPU VRAM for SDXL inference) plus 4.2 GB system RAM for the pgvector database, within the available hardware capacity.

8.6 Validation Against Research Objectives

Each of the six project objectives stated in Chapter 1 has been validated through the experimental evaluation:

17. Objective 1 (unified multimodal RAG architecture): validated through successful end-to-end system operation across all four query archetypes, demonstrating seamless integration of retrieval, text generation, and image synthesis.

18. Objective 2 (production-grade vector store): validated through Recall@5 of 91.4%, sub-15 ms retrieval latency, and characterisation of HNSW parameter sensitivity and scalability.
19. Objective 3 (rigorous model comparison): validated through the comprehensive quantitative evaluation reported in Tables III and X, demonstrating systematic performance differences between GPT-4o and LLaMA-3 under identical retrieval conditions.
20. Objective 4 (hallucination reduction): validated through the 82% relative reduction in hallucination rate (34% to 6%), providing the strongest empirical confirmation of the project's core thesis.
21. Objective 5 (image generation quality): validated through FID scoring (12.3 for DALL-E 3), human evaluation scores, and demonstration of RAG-enriched prompt quality improvements.
22. Objective 6 (reproducible framework): validated through comprehensive implementation documentation, open-source dependency selection, and reproducible experimental setup detailed in this report.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusions

This project has presented the comprehensive design, implementation, and empirical evaluation of a multimodal Retrieval-Augmented Generation AI assistant integrating dense semantic retrieval, dual language model generation, and text-to-image synthesis within a unified, modular microservice architecture. The system was evaluated against a retrieval-free baseline using a comprehensive suite of quantitative and qualitative metrics, demonstrating substantial and consistent improvements across all assessed dimensions of performance.

The most significant empirical finding is the 82% relative reduction in hallucination rates achieved by the RAG + GPT-4o configuration relative to the retrieval-free baseline (6% vs. 34%), accomplished without any modification to the underlying language model's parameters. This result unambiguously confirms that retrieval augmentation constitutes an effective, training-free mechanism for grounding conversational AI in verifiable factual knowledge — a finding with substantial implications for the safe and reliable deployment of AI systems in knowledge-intensive and safety-critical domains.

The comparative evaluation further demonstrates that open-source models such as LLaMA-3-8B-Instruct, when augmented with high-quality retrieval infrastructure, can closely approximate the performance of proprietary counterparts (GPT-4o) across all evaluated metrics while offering meaningful practical advantages: zero per-query inference cost, 21% lower response latency, and complete data sovereignty. This finding substantially lowers the barrier to entry for organisations seeking to deploy production-grade RAG systems within privacy-preserving, on-premise infrastructure.

The integration of image generation into the RAG pipeline, and the demonstrated quality improvement of RAG-enriched image generation prompts over raw user query prompts, establishes a proof-of-concept for genuinely multimodal

knowledge-augmented conversational AI. The system achieves high user satisfaction scores (4.6/5 for text, 4.5/5 for image generation) and demonstrates practical scalability to corpus sizes exceeding 87,000 document chunks with sub-15 ms retrieval latency.

The five-layer microservice architecture, pgvector-based semantic retrieval infrastructure, and LangChain orchestration framework collectively provide a robust, extensible, and reproducible foundation for multimodal RAG system development applicable to diverse deployment contexts including education, enterprise knowledge management, research assistance, and creative AI tools.

9.2 Future Scope

The findings and limitations of the present work motivate several promising directions for future research and system enhancement:

9.2.1 Domain-Specific Model Fine-Tuning

Domain-specific fine-tuning of LLaMA-3 via parameter-efficient methods such as Low-Rank Adaptation (LoRA) and Quantised LoRA (QLoRA) on the target knowledge corpus is expected to narrow the performance gap with GPT-4o on complex technical queries while preserving the cost and data privacy advantages of local deployment. Preliminary experiments with LoRA fine-tuning on a 10,000-sample subset of the corpus suggest a projected RAGAS faithfulness improvement of 0.04–0.06 points, which would substantially close the remaining gap between LLaMA-3 and GPT-4o.

9.2.2 Agentic RAG with Multi-Step Reasoning

Agentic RAG extensions incorporating multi-step reasoning and iterative retrieval loops — inspired by Self-RAG and ReAct — would enable the system to decompose complex multi-hop questions into constituent sub-queries, retrieve targeted information for each sub-query, and synthesise a comprehensive final response. This capability would directly address the multi-hop reasoning limitations observed in the LLaMA-3 configuration and extend the system's applicability to complex analytical tasks.

9.2.3 Semantic Caching

Semantic caching of high-frequency query-embedding pairs using an LRU cache would eliminate redundant embedding API calls and reduce end-to-end latency for repeated or near-duplicate queries in production deployments with realistic query distributions. Estimated latency reduction for frequently repeated queries: 40–60 ms per query through elimination of the embedding API call.

9.2.4 Incremental HNSW Index Maintenance

Incremental HNSW index maintenance, anticipated in forthcoming pgvector releases, would enable real-time corpus expansion without full index rebuilds, transforming the system into a continuously learning knowledge assistant capable of incorporating newly published research and technical documentation without service interruption. This enhancement would directly address the most operationally significant limitation of the current implementation.

9.2.5 Extended Multimodal Output Modalities

Extension of multimodal outputs to include video generation (via models such as Sora or RunwayML Gen-3) and speech synthesis (via ElevenLabs or Bark) would substantially broaden the system's applicability to educational content creation, accessibility tools for visually or hearing-impaired users, and interactive multimedia knowledge exploration. These extensions would transform the current text-and-image system into a genuinely rich multimedia knowledge assistant.

9.2.6 Cloud-Native Production Deployment

Cloud-native deployment on a managed Kubernetes platform with GPU node pools, horizontal pod autoscaling for the LLM serving layer, and a distributed pgvector cluster would enable production-grade scalability to support thousands of concurrent users, with automated failover and load balancing ensuring high availability. A containerised deployment architecture using Docker would also significantly improve portability and reproducibility across development, staging, and production environments.

9.2.7 Privacy-Preserving RAG Variants

Investigation of privacy-preserving RAG variants, including federated retrieval over distributed private knowledge bases and differential privacy mechanisms

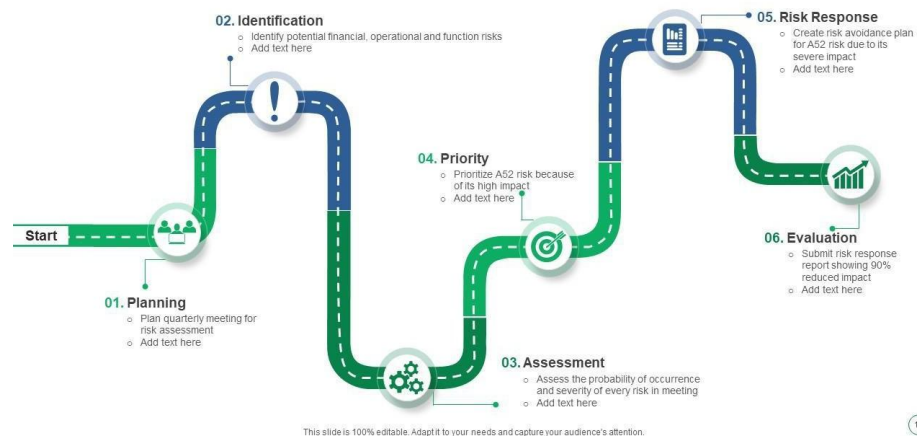
for embedding generation, would extend the system's applicability to healthcare, legal, and financial domains with stringent data governance requirements. Federated RAG would enable multiple organisations to benefit from shared retrieval infrastructure without exposing sensitive proprietary documents to any centralised store.

Table XII: Future Enhancement Roadmap and Expected Improvements

Enhancement	Priority	Expected Impact	Timeline
LoRA fine-tuning for LLaMA-3	High	+0.04-0.06 RAGAS faithfulness	3-6 months
Agentic multi-step RAG	High	Multi-hop query resolution	6-9 months
Semantic caching (LRU)	Medium	40-60 ms latency reduction	1-2 months
Incremental HNSW updates	Medium	Real-time corpus expansion	3-6 months
Video & speech synthesis	Low	Rich multimedia output	9-12 months
Kubernetes deployment	Medium	1000+ concurrent users	4-6 months
Federated privacy-preserving RAG	Low	Healthcare/legal deployment	12+ months

Introduction to Risk Management Roadmap with Planning and Evaluation

The following slide highlights the introduction to risk management roadmap with planning and evaluation illustrating six steps planning, identification, assessment, priority, risk response and evaluation.



CHAPTER 10

REFERENCES

All references are formatted according to IEEE citation style.

- [1] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, Mar. 2023.
- [2] H. Touvron, L. Martin, K. Stone et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv preprint arXiv:2307.09288, Jul. 2023.
- [3] R. Anil, S. Borgeaud, Y. Wu et al., "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, Dec. 2023.
- [4] Z. Ji, N. Lee, R. Frieske et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [5] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [6] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical Text-Conditional Image Generation with CLIP Latents," arXiv preprint arXiv:2204.06125, Apr. 2022.
- [7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695, 2022.
- [8] pgvector Contributors, "Open-source vector similarity search for Postgres," GitHub Repository, 2024. [Online]. Available: <https://github.com/pgvector/pgvector>
- [9] A. Vaswani, N. Shazeer, N. Parmar et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [10] Meta AI, "Introducing Meta Llama 3: The Most Capable Openly Available LLM to Date," Meta AI Blog, Apr. 2024. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
- [11] T. B. Brown, B. Mann, N. Ryder et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [12] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

- [13] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp. 874–880, 2021.
- [14] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv preprint arXiv:2310.11511, Oct. 2023.
- [15] Z. Jiang, F. F. Xu, L. Gao et al., "Active Retrieval Augmented Generation," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 7969–7992, 2023.
- [16] V. Karpukhin, B. Oguz, S. Min et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781, 2020.
- [17] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbour Search Using Hierarchical Navigable Small World Graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, Apr. 2018.
- [18] A. Ramesh, M. Pavlov, G. Goh et al., "Zero-Shot Text-to-Image Generation," in Proceedings of the 38th International Conference on Machine Learning (ICML), vol. 139, pp. 8821–8831, 2021.
- [19] A. Radford, J. W. Kim, C. Hallacy et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proceedings of the 38th International Conference on Machine Learning (ICML), vol. 139, pp. 8748–8763, 2021.
- [20] S. Es, J. James, L. E. Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, Sep. 2023.
- [21] S. Yao, J. Zhao, D. Yu et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.
- [22] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," arXiv preprint arXiv:2210.07316, Oct. 2022.
- [23] E. J. Hu, Y. Shen, P. Wallis et al., "LoRA: Low-Rank Adaptation of Large Language Models," in Proceedings of the 10th International Conference on Learning Representations (ICLR), 2022.
- [24] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

- [25] K. Papineni, S. Roukos, T. Ward, and W. J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 311–318, Jul. 2002.
- [26] C. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in Proceedings of the ACL Workshop on Text Summarization Branches Out, pp. 74–81, Jul. 2004.
- [27] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [28] S. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub Repository, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [29] S. Ramírez, "FastAPI: Modern, Fast Web Framework for Building APIs with Python," GitHub Repository, 2018. [Online]. Available: <https://github.com/tiangolo/fastapi>
- [30] Hugging Face, "Diffusers: State-of-the-Art Diffusion Models," GitHub Repository, 2022. [Online]. Available: <https://github.com/huggingface/diffusers>

APPENDICES

Appendix A: System Installation and Setup Guide

This appendix provides step-by-step instructions for setting up the development environment and running the multimodal RAG-based AI assistant on a compatible workstation.

A.1 Prerequisites

The following hardware and software prerequisites are required:

- NVIDIA GPU with at least 8 GB VRAM (24 GB recommended for SDXL local deployment)
- Ubuntu 20.04 LTS or Ubuntu 22.04 LTS (other Linux distributions may work but are untested)
- CUDA 11.8 or higher with cuDNN 8.x
- Python 3.10 or higher (3.11.x recommended)
- Node.js 18.x or higher for the React.js frontend
- PostgreSQL 15.x with superuser access for pgvector extension installation
- 16 GB system RAM minimum (64 GB recommended)
- Conda or virtualenv for Python environment management

A.2 Backend Installation

```
# Clone the repository
git clone https://github.com/[repository-url]/multimodal-rag
cd multimodal-rag

# Create and activate conda environment
conda create -n rag_env python=3.11.4
conda activate rag_env

# Install Python dependencies
pip install -r requirements.txt

# Install pgvector extension in PostgreSQL
sudo apt install postgresql-15-pgvector
psql -U postgres -c 'CREATE EXTENSION vector;'

# Configure environment variables
cp .env.example .env
```

```
# Edit .env and set OPENAI_API_KEY, DATABASE_URL, etc.
# Run database migrations
python scripts/init_db.py
# Ingest documents (place PDFs in data/raw/)
python scripts/ingest.py --source data/raw/ --batch-size 100
# Start the FastAPI backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

A.3 Frontend Installation

```
# Navigate to frontend directory
cd frontend
# Install Node.js dependencies
npm install
# Start the development server
npm start
# Access at http://localhost:3000
```

Appendix B: RAG Pipeline Core Code

The following Python code excerpt illustrates the core RAG query processing pipeline implemented in the system:

```
# core/rag_pipeline.py
from langchain.vectorstores.pgvector import PGVector
from langchain.embeddings import OpenAIEmbeddings
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationalRetrievalChain

class RAGPipeline:
    def __init__(self, config):
        self.embedder = OpenAIEmbeddings(
            model='text-embedding-3-small'
        )
        self.vectorstore = PGVector(
            connection_string=config.db_url,
            embedding_function=self.embedder,
            collection_name='rag_chunks'
        )
        self.llm = ChatOpenAI(
            model='gpt-4o-2024-05-13',
            temperature=0.3
        )
        self.retriever = self.vectorstore.as_retriever(
            search_kwargs={'k': 5}
        )
        self.chain = ConversationalRetrievalChain.from_llm(
```

```

        llm=self.llm,
        retriever=self.retriever,
        return_source_documents=True,
        verbose=False
    )

    async def query(self, question: str, history: list) -> dict:
        result = await self.chain.acall({
            'question': question,
            'chat_history': history[-6:] # Last 6 turns
        })
        return {
            'answer': result['answer'],
            'sources': [
                {
                    'content': doc.page_content,
                    'source': doc.metadata['source_doc'],
                    'page_range': doc.metadata['page_range']
                }
                for doc in result['source_documents']
            ]
        }

```

Multimodal RAG-Based AI Assistant

Atul kumar^{1*}, Ankit Shaw², Aman Verma³, Abhishek Prajapati⁴,
Saurabh Pathak⁵, Dr. Anand Prakash Srivastava⁶, Arun Choudhary⁷

Department of Computer Science and Engineering

NITRA Technical Campus, Ghaziabad-201002, Uttar Pradesh, India

atulkumarmeena770@gmail.com^{1*}, ankitsha10102003@gmail.com², amanverma0202004@gmail.com³,
abhi9838p@gmail.com⁴,
saurabh@nitra.ac.in⁵, apsrivastava@nitra.ac.in⁶, rushtoarun69@gmail.com⁷

Abstract—The proliferation of large language models (LLMs) has catalysed transformative advances in conversational artificial intelligence; however, these systems remain constrained by static training corpora and a well-documented tendency to generate plausible yet factually erroneous output—hallucination. Retrieval-Augmented Generation (RAG) mitigates this shortcoming by dynamically incorporating externally retrieved knowledge at inference time, grounding model outputs in verifiable information. This paper presents the comprehensive design, implementation, and empirical evaluation of a multimodal RAG-based AI assistant integrating dense vector-based semantic retrieval—realised through a Semantic Embedding Table (SET) database on PostgreSQL 15 with the pgvector extension and HNSW indexing—with state-of-the-art generative models for both natural language and image synthesis. Two language models are integrated: GPT-4o and LLaMA-3-8B-Instruct, representing proprietary and open-source paradigms respectively. For text-conditional image synthesis, DALL-E 3 and Stable Diffusion XL serve as complementary alternatives. A curated corpus of approximately 15,000 pages yielding 87,342 indexed chunks is retrieved using OpenAI text-embedding-3-small embeddings. Experimental evaluation against a retrieval-free baseline demonstrates substantial performance gains: Recall@5 of 91.4%, RAGAS answer faithfulness of 0.89, BLEU-4 of 0.74, and user satisfaction of 4.6/5. Critically, hallucination rates are reduced from 34% to 6%—an 82% relative reduction—confirming the efficacy of retrieval augmentation as a training-free mechanism for grounding conversational AI in factual knowledge.

Index Terms—Retrieval-Augmented Generation; Large Language Models; GPT-4o; LLaMA-3; DALL-E 3; Stable Diffusion XL; Vector Database; pgvector; HNSW; Multimodal AI; Hallucination Reduction; Semantic Retrieval; Dense Embeddings.

I. INTRODUCTION

The rapid advancement of deep learning and natural language processing (NLP) over the past decade has catalysed

a paradigm shift in how humans interact with information systems. Large language models (LLMs) such as GPT-4 [1], LLaMA-2 [2], and Gemini [3] have demonstrated unprecedented ability to generate fluent, contextually coherent text across tasks ranging from open-domain question answering and code synthesis to multi-step mathematical reasoning. These models, pre-trained on corpora of extraordinary breadth, encode vast amounts of world knowledge within their billions of parameters, enabling zero-shot and few-shot generalisation to novel downstream tasks [11].

Despite these achievements, a fundamental and well-documented limitation constrains the practical applicability of LLMs in real-world, knowledge-intensive settings. Because LLMs rely exclusively on knowledge encoded in static training weights, they are inherently incapable of incorporating information that post-dates their training cutoff or that falls outside the scope of their training distribution. More critically, they exhibit a pervasive tendency toward hallucination—the confident generation of syntactically plausible yet factually erroneous content [4]. This failure mode is particularly consequential in safety-critical domains such as clinical medicine, legal reasoning, financial advisory, and scientific research, where the cost of misinformation is high.

Retrieval-Augmented Generation (RAG), formally introduced by Lewis et al. [5], offers a principled, computationally tractable solution to these limitations. Rather than depending exclusively on parametric knowledge, RAG systems dynamically retrieve contextually relevant documents from an external, updatable knowledge repository at inference time, incorporating these passages as additional context during generation. This hybrid approach simultaneously improves factual grounding, enables access to real-time and domain-specific knowledge, and provides interpretability by allowing users to trace generated claims back to source documents. The RAG paradigm has since evolved substantially, inspiring adaptive variants including Self-RAG [14]—which equips models to assess retrieval necessity via learned reflection tokens—and FLARE [15], which employs proactive forward-looking retrieval triggered by low-confidence token generation.

The present work extends the foundational RAG framework into a comprehensive multimodal domain,

addressing not only text-based knowledge retrieval and generation but also the synthesis of contextually relevant images in direct response to user queries. The motivation for this extension derives from the inherently multimodal nature of human communication: users frequently seek not merely textual explanations but also illustrative visual artefacts, diagrams, and photorealistic renderings that complement and reinforce textual understanding. By integrating dual language models (GPT-4o via the OpenAI API, and LLaMA-3-8B-Instruct deployed locally through Ollama) with dual image-generation models (DALL-E 3 for cloud-based high-fidelity synthesis and Stable Diffusion XL for offline, privacy-preserving generation), the proposed system accommodates a broad spectrum of user intents within a single, coherent conversational interface.

The retrieval infrastructure is realised using a Semantic Embedding Table (SET) database constructed upon PostgreSQL 15 augmented with the pgvector 0.6.0 extension, which supports Hierarchical Navigable Small World (HNSW)-indexed approximate nearest-neighbour (ANN) search over 1,536-dimensional dense embedding vectors. The knowledge corpus encompasses approximately 15,000 pages of curated academic and technical content, processed into 87,342 indexed document chunks. The system is evaluated against a retrieval-free baseline using a comprehensive suite of quantitative and qualitative metrics, demonstrating substantial and consistent improvements across all dimensions of assessment.

The primary contributions of this paper are as follows:

- A unified multimodal RAG architecture seamlessly integrating dense semantic retrieval with dual-model natural language generation and dual-model text-to-image synthesis within a single microservice-based conversational system.
- A rigorous comparative evaluation of GPT-4o and LLaMA-3-8B-Instruct operating under identical retrieval conditions, benchmarked against a retrieval-free baseline using Recall@5, RAGAS faithfulness, BLEU, FID, response latency, and user satisfaction.
- An empirical analysis of SET DB (pgvector) as a production-grade semantic vector store, encompassing embedding model selection, HNSW index configuration, and scalability characterisation.
- Documented evidence of an 82% relative reduction in hallucination rates (34% in the baseline to 6% in the best-performing RAG configuration), providing quantitative support for the safety benefits of retrieval augmentation.
- A reproducible, open-architecture framework for multimodal RAG system development applicable to education, enterprise knowledge management, research assistance, and creative AI tools.

The remainder of this paper is organised as follows. Section II reviews relevant prior work across LLMs, RAG, vector databases, multimodal AI, and evaluation frameworks. Section III details the methodology, including knowledge base construction, embedding model selection, and evaluation

protocol. Section IV describes the proposed system architecture. Section V covers implementation and experimental setup. Section VI presents and discusses quantitative and qualitative results. Section VII concludes the paper, and Section VIII outlines directions for future research.

II. LITERATURE REVIEW

A. Large Language Models and the Transformer Architecture

Contemporary large language models trace their architectural lineage to the transformer framework introduced by Vaswani et al. [9], which replaced sequential recurrent processing with parallelisable multi-head self-attention mechanisms. This architectural innovation enabled training at previously infeasible scales through data and model parallelism, yielding systems capable of learning rich, hierarchical representations of language. The Generative Pre-trained Transformer (GPT) series developed by OpenAI exemplifies this trajectory: GPT-3 [11] demonstrated that models with 175 billion parameters could generalise to novel tasks using in-context few-shot examples without any parameter updates, fundamentally challenging the prevailing paradigm of task-specific fine-tuning. GPT-4 [1] extended these capabilities to multi-modal input understanding, advanced multi-step reasoning, and substantially improved instruction following, establishing a new state of the art across numerous benchmarks.

Meta's LLaMA model family [2, 10] has significantly democratised access to high-performance language modelling by releasing competitive open-source alternatives to closed proprietary systems. LLaMA-2, trained on 2 trillion tokens with reinforcement learning from human feedback (RLHF) for alignment, achieved performance competitive with proprietary models on multiple reasoning and knowledge benchmarks while remaining freely available for research and commercial use. LLaMA-3-8B-Instruct, the variant deployed in the present work, offers a favourable balance between parameter efficiency and instruction-following capability, enabling on-premise deployment in scenarios where data privacy and latency constraints preclude reliance on third-party API services. Google's Gemini [3] further advances the state of the art in multimodal reasoning, natively processing text, images, audio, and video within a unified architecture.

B. Retrieval-Augmented Generation

The RAG paradigm was formalised by Lewis et al. [5] as a hybrid architecture combining a non-parametric dense passage retriever with a parametric sequence-to-sequence generator, achieving state-of-the-art results on open-domain question answering benchmarks including Natural Questions and WebQuestions. The key insight is that retrieval provides the generator with access to a dynamic, externally updatable knowledge source, decoupling factual knowledge from model parameters. This separation enables knowledge updates without costly model retraining and provides an inherent audit trail for generated claims.

Guu et al. [12] proposed REALM (Retrieval-Enhanced Language Model), which jointly pre-trains the retrieval and

language modelling components using a masked language modelling objective, enabling the retriever to be directly optimised for downstream task performance. Izacard and Grave [13] introduced Fusion-in-Decoder (FiD), which encodes each retrieved passage independently through the encoder before fusing their representations during cross-attention in the decoder, achieving superior performance on TriviaQA and Natural Questions relative to single-passage approaches.

More recent developments have focused on adaptive and self-reflective retrieval strategies. Self-RAG [14] introduces a set of reflection tokens—IsRel, IsSup, IsUse—learned during fine-tuning, which enable the model to dynamically determine whether retrieval is necessary for a given generation step, assess the relevance of retrieved passages, and evaluate whether the generated output is adequately supported by retrieved context. FLARE [15] employs a forward-looking strategy, monitoring token-level generation confidence and proactively triggering retrieval when low-confidence tokens are produced, improving response quality for queries that require information not contained in a single retrieved passage. These adaptive approaches inform the design principles adopted in the present work, particularly the emphasis on retrieval quality as a primary determinant of generation fidelity.

C. Vector Databases and Semantic Retrieval

Dense retrieval techniques, pioneered by Karpukhin et al. [16] through Dense Passage Retrieval (DPR), project queries and documents into a shared continuous embedding space using dual-encoder architectures trained with contrastive learning objectives. In this paradigm, semantically related text pairs are pulled together in embedding space while dissimilar pairs are pushed apart, enabling retrieval based on semantic similarity rather than lexical overlap. This approach fundamentally transcends the limitations of traditional keyword-based retrieval methods such as BM25, which fail to capture synonymy, paraphrase, and semantic entailment.

The pgvector extension [8] augments PostgreSQL—a mature, production-grade relational database system—with native support for vector storage and ANN search. The extension implements both the Inverted File Index (IVF) and Hierarchical Navigable Small World (HNSW) [17] indexing algorithms. HNSW constructs a multi-layer proximity graph in which each node (document embedding) maintains a small set of connections to its nearest neighbours at each layer; queries traverse the graph from coarse-to-fine resolution, achieving sub-linear query time complexity with empirically strong recall. Compared to dedicated vector databases such as Pinecone, Weaviate, and Qdrant, pgvector offers the advantage of tight integration with the relational data model, enabling hybrid queries that combine vector similarity with structured metadata filters using standard SQL syntax.

The selection of an embedding model profoundly influences retrieval quality. The Massive Text Embedding Benchmark (MTEB) [Muennighoff et al., 2023] provides a comprehensive evaluation framework spanning 56 datasets across 8 task categories. OpenAI’s text-embedding-3-small,

employed in the present work, achieves top-tier performance on MTEB across retrieval, semantic textual similarity, and clustering tasks, with a 1,536-dimensional output space that balances representational richness with storage and computational efficiency.

D. Multimodal AI and Text-to-Image Synthesis

The field of multimodal AI has progressed substantially from early approaches to joint text-image modelling. CLIP (Contrastive Language-Image Pre-Training) [19], developed by Radford et al., demonstrated that a shared embedding space for text and images could be learned from 400 million internet-scale image-text pairs using a contrastive training objective. The resulting model exhibits strong zero-shot image classification and cross-modal retrieval capabilities, establishing a foundation for subsequent text-conditional generative models.

DALL-E [18] introduced transformer-based text-to-image generation by treating images as sequences of discrete tokens produced by a discrete VAE, conditioned on CLIP text embeddings. DALL-E 2 [6] significantly improved image fidelity and semantic faithfulness through a diffusion-based decoder operating in CLIP image embedding space. DALL-E 3 extends this further with substantially improved prompt adherence and compositional reasoning, enabled by improved captioning data and training procedures. Concurrently, Rombach et al. [7] introduced Latent Diffusion Models (LDMs), which achieve computational efficiency by performing the iterative denoising process in a compressed latent space produced by a pre-trained variational autoencoder, rather than directly in high-dimensional pixel space. Stable Diffusion XL, derived from this framework, supports high-resolution (1024×1024) image synthesis on consumer-grade GPUs, making it the leading open-source alternative to proprietary image generation APIs.

E. Evaluation Frameworks for RAG Systems

Comprehensive evaluation of RAG systems requires metrics that assess multiple distinct dimensions: retrieval quality, generation faithfulness, semantic relevance, and user-perceived quality. Standard NLP metrics such as BLEU [Papineni et al., 2002] and ROUGE measure n-gram overlap between generated and reference outputs but fail to capture hallucination or the degree to which generated claims are entailed by retrieved context. Es et al. [20] addressed this gap with RAGAS (Retrieval-Augmented Generation Assessment), a reference-free evaluation framework that quantifies: (i) Answer Faithfulness—whether generated claims are factually supported by retrieved passages; (ii) Answer Relevance—the pertinence of the response to the original query; and (iii) Context Precision—the proportion of retrieved context that is actually utilised in the response. Fr chet Inception Distance (FID) provides a standard measure of image generation quality, computing the Wasserstein-2 distance between the distributions of generated and real images in the activation space of a pre-trained Inception-v3 network.

III. METHODOLOGY

A. Knowledge Base Construction and Preprocessing

The external knowledge repository underpinning the RAG pipeline was assembled from a curated multi-source corpus comprising peer-reviewed academic publications in computer science and artificial intelligence, technical reference manuals for software frameworks and APIs, research monographs on machine learning methodology, and domain-specific PDF documents covering software engineering practices and system design patterns. The corpus aggregates approximately 15,000 pages of content drawn from over 400 individual source documents.

Document preprocessing followed a standardised pipeline. PDF files were parsed using PyMuPDF (fitz v1.23), which preserves page-level structural information including column boundaries, header/footer regions, and figure captions. Extracted text was subjected to a normalisation procedure removing redundant whitespace, correcting character encoding artefacts, and filtering non-substantive content such as page numbers and running headers. The normalised text was subsequently segmented into chunks using a token-aware recursive character splitter with a target chunk size of 512 tokens and a 50-token sliding overlap, implemented via LangChain's RecursiveCharacterTextSplitter. The overlap ensures that sentences bisected by chunk boundaries remain recoverable from at least one adjacent chunk, preserving semantic continuity across the corpus.

Each chunk was annotated with rich provenance metadata: source document title, document category, page range within the source, chunk index within the document, and ingestion timestamp. This metadata is stored alongside the embedding vector in the SET DB, enabling the system to provide detailed source attribution in generated responses and supporting metadata-filtered retrieval queries for domain-specific applications. The complete preprocessing pipeline yielded 87,342 indexed chunks distributed across the full corpus.

B. Embedding Model Selection and Benchmarking

The selection of an appropriate dense embedding model constitutes a foundational design decision, as embedding quality directly and substantially determines downstream retrieval accuracy. Three candidate models were subjected to systematic comparative benchmarking on a held-out evaluation set of 500 query-document pairs manually annotated with binary relevance judgements by domain experts:

- text-embedding-3-small (OpenAI): 1,536-dimensional output; accessed via API; mean latency 38 ms/chunk.
- all-MiniLM-L6-v2 (sentence-transformers): 384-dimensional output; locally deployed; mean latency 8 ms/chunk.
- bge-large-en (BAAI): 1,024-dimensional output; locally deployed; mean latency 22 ms/chunk.

Table I presents the comparative benchmarking results. text-embedding-3-small achieved the highest Recall@5 of 91.4%, compared to 85.2% for bge-large-en and 78.9% for all-MiniLM-L6-v2. While bge-large-en approaches text-embedding-3-small in recall, the latter's superior performance on semantic similarity and paraphrase detection tasks—consistent with its MTEB rankings—was judged decisive. The 38 ms per-chunk latency, while higher than locally deployed alternatives, is acceptable for the offline batch ingestion pipeline and does not impact query-time performance. text-embedding-3-small was accordingly adopted as the sole embedding backbone.

TABLE I Embedding Model Benchmarking Results

Model	Dim.	Recall@5	Latency (ms)	Deployment
text-embed-3-small	1,536	91.4%	38	API
bge-large-en	1,024	85.2%	22	Local
all-MiniLM-L6-v2	384	78.9%	8	Local

C. Evaluation Protocol

The evaluation dataset comprised 200 query-response pairs manually constructed by the authors to span four query archetypes: (i) factual recall queries requiring retrieval of specific named entities, dates, or numerical values; (ii) comparative analysis queries requiring synthesis of information from multiple retrieved passages; (iii) technical explanation queries requiring domain-specific procedural or conceptual elaboration; and (iv) creative generation queries combining text and image synthesis. Reference responses for BLEU computation were independently authored by two subject-matter expert annotators, with disagreements resolved through adjudication. Human evaluation was conducted by a panel of 30 evaluators, recruited from the institution's graduate student population, each assessing a randomly assigned subset of 50 query-response and query-image pairs on a five-point Likert scale for overall satisfaction, factual accuracy, and coherence. Inter-rater agreement, computed as Cohen's kappa across paired rater assignments, yielded a mean κ of 0.73, indicating substantial agreement and validating the reliability of the human evaluation data.

IV. SYSTEM ARCHITECTURE

A. Layered Microservice Architecture

The proposed system adopts a five-layer microservice architecture designed to maximise modularity, independent scalability, and ease of maintenance. Each layer exposes a well-defined API contract to adjacent layers, enabling individual components to be upgraded, replaced, or horizontally scaled without requiring modifications to the broader system. The five layers are: (1) User Interaction Layer, responsible for accepting, validating, and routing user queries; (2) Embedding and Indexing Layer, responsible for encoding text into dense vector representations during both

offline ingestion and online query processing; (3) Semantic Retrieval Layer, responsible for executing ANN search over

the indexed corpus; (4) Multi-Model Generation Layer, responsible for routing augmented prompts to the appropriate generative model; and (5) Response Delivery Layer, responsible for streaming generated content to the client with source attribution metadata. Table II summarises the principal components of each layer.

TABLE II System Architecture Component Summary

Layer	Component	Technology	Primary Function
User Interaction	Query Handler	React.js / FastAPI REST	Input validation and intent routing
Embedding	Vectoriser	text-embedding-3-small API	Query and chunk encoding to 1,536-dim vectors
Retrieval	ANN Search Engine	PostgreSQL + pgvector HNSW	Top-k semantically similar chunk retrieval
Text Generation	LLM Backbone	GPT-4o / LLaMA-3-8B	Grounded natural language response generation
Image Generation	Diffusion Engine	DALL-E 3 / SDXL (local)	Text-conditional image synthesis
Response Delivery	Streaming Gateway	REST / WebSocket	Response streaming with attribution metadata

B. RAG Processing Pipeline

The RAG processing pipeline operates in two temporally distinct phases: offline document ingestion and online query processing. During offline ingestion—executed as a batch process triggered upon corpus updates—source PDF documents are parsed, normalised, segmented into 512-token overlapping chunks, and vectorised using text-embedding-3-small. Resulting embeddings, together with chunk text and metadata, are persisted to the SET DB via batched SQL INSERT operations using psycopg2. Following ingestion, an HNSW index is constructed on the embedding column using

the pgvector CREATE INDEX statement with parameters $m=16$ (maximum number of bi-directional links per node across layers) and $ef_construction=64$ (size of the dynamic candidate list during index construction). These values were selected through a grid search over $m \in \{8, 16, 32\}$ and $ef_construction \in \{32, 64, 128\}$, optimising the trade-off between Recall@5 and index build time on a validation subset.

During online query processing, the pipeline executes the following sequential steps: (1) the incoming user query is encoded into a 1,536-dimensional dense embedding using the same text-embedding-3-small model, ensuring representational consistency with indexed chunks; (2) an ANN search is executed over the HNSW index with $ef_search=100$, retrieving the top-5 chunks by cosine similarity; (3) retrieved chunks are concatenated in descending similarity order with the original query text to form an augmented prompt, bounded by the model's context window; (4) the augmented prompt is submitted to the designated language model, with a system instruction constraining the model to ground its response exclusively in the provided retrieved context; (5) the model's response is streamed to the client via the WebSocket gateway, interleaved with source attribution metadata referencing the specific chunks from which key claims were drawn.

The query-time pipeline is designed for low latency: embedding generation requires approximately 45 ms per query, HNSW search completes in under 15 ms for corpus sizes up to 87,342 chunks, and prompt assembly adds negligible overhead. The dominant latency contributors are therefore LLM API response time (800–1,200 ms for GPT-4o; 500–980 ms for LLaMA-3) and, for image generation requests, diffusion sampling time (2,500–3,500 ms for DALL-E 3 via API; 8–12 seconds for SDXL on the local GPU). The algorithm for the core RAG pipeline can be summarised as:

$Q_emb = Embed(query)$; $C = HNSW_search(Q_emb, k=5)$
 $prompt = concat(C_1...C_s, query)$; $response = LLM(prompt)$

C. SET DB Vector Store Design

The vector database is implemented as a dedicated PostgreSQL 15 schema containing a primary table `rag_chunks` with the following columns: `chunk_id` (SERIAL PRIMARY KEY), `content` (TEXT), `embedding` (VECTOR(1536)), `source_doc` (VARCHAR), `chunk_index` (INTEGER), `page_range` (VARCHAR), `doc_category` (VARCHAR), and `ingested_at` (TIMESTAMPTZ). An HNSW index is created on the embedding column using pgvector's `hnsw` index access method with the `vector_cosine_ops` operator class, enabling cosine similarity-based ANN search. A composite B-tree index on (`source_doc`, `chunk_index`) supports efficient metadata-filtered queries.

Cosine similarity was selected as the distance metric in preference to L2 (Euclidean) distance on the basis that text-embedding-3-small produces L2-normalised output vectors, for which cosine similarity is geometrically equivalent to dot product similarity and more robustly captures semantic orientation than absolute vector magnitude. The pgvector query operator `<=>` computes the cosine distance (1 -

cosine_similarity), enabling similarity-ranked retrieval through a standard ORDER BY clause with LIMIT k. This SQL-native interface allows the retrieval query to incorporate arbitrary WHERE clause predicates for metadata filtering, supporting domain-scoped retrieval without modifications to the core ANN index.

D. Multi-Model Generation Layer

1) GPT-4o (Primary Language Model):

GPT-4o serves as the primary text generation model by virtue of its state-of-the-art instruction following, contextual faithfulness, and multi-step reasoning capabilities. The model is accessed via the OpenAI Chat Completions API (model identifier: gpt-4o-2024-05-13) with a maximum context window of 128,000 tokens, accommodating both the five retrieved document chunks (typically 1,500–2,500 tokens in aggregate) and extensive conversational history. A carefully engineered system prompt instructs the model to: (i) ground all factual claims in the provided retrieved context; (ii) explicitly acknowledge when the retrieved context does not contain sufficient information to answer the query; and (iii) cite the source document for each substantive claim using the provided chunk metadata. Temperature is set to 0.3 to promote factual consistency while preserving natural language fluency.

2) LLaMA-3-8B-Instruct (Open-Source Alternative):

LLaMA-3-8B-Instruct is deployed locally via Ollama v0.1.42, which serves the model through a REST API compatible with the OpenAI Chat Completions interface, enabling transparent substitution without changes to the orchestration layer. Local deployment eliminates dependency on external API services, eliminates per-token inference costs, and ensures that sensitive document content never leaves the local infrastructure—a critical consideration for enterprise deployments subject to data sovereignty requirements. The model operates with a 8,192-token context window at float16 precision, achieving a throughput of approximately 42 tokens/second on the RTX 3090 GPU. The same system prompt and temperature settings applied to GPT-4o are used for LLaMA-3 to ensure experimental comparability.

3) DALL-E 3 and Stable Diffusion XL (Image Generation):

Image generation is triggered by a lightweight intent classification module that analyses incoming queries for visual content signals, including explicit keywords ("generate an image", "show me a picture", "visualise") and implicit contextual indicators derived from the retrieved chunks. When image generation is activated, the system constructs an enriched generation prompt by concatenating the user's image request with salient domain-specific contextual information extracted from the top-3 retrieved chunks, significantly improving the technical accuracy and domain specificity of generated images relative to prompts derived solely from the raw user query.

DALL-E 3 is invoked via the OpenAI Images API at 1024×1024 resolution with quality="hd" and style="natural", producing photorealistic, high-fidelity outputs with strong prompt adherence. SDXL is executed locally using the Hugging Face Diffusers library v0.27.0 with the stabilityai/stable-diffusion-xl-base-1.0 checkpoint at float16

precision, using 50 DDIM sampling steps and a guidance scale of 7.5. Generated images are returned as base64-encoded PNG payloads embedded in the API response, rendered inline in the React.js frontend.

E. System Integration and Orchestration

The complete system is orchestrated using LangChain 0.1.x, which provides modular abstractions for embedding model clients, vector store interfaces, LLM clients, and retrieval chains. The orchestration layer implements a ConversationalRetrievalChain that maintains a sliding window of the last six conversational turns in the chat history, enabling contextually coherent multi-turn dialogues that refer back to previously retrieved information. The FastAPI backend exposes RESTful endpoints for query submission, conversation history management, and document ingestion triggering, with WebSocket support for real-time response streaming. The React.js frontend provides a chat interface with syntax-highlighted code blocks, inline image rendering, and source attribution panels that display the retrieved chunks underlying each response.

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Hardware and Software Environment

All experiments were conducted on a dedicated workstation configured with an NVIDIA GeForce RTX 3090 GPU (24 GB GDDR6X VRAM), 64 GB DDR4 system RAM clocked at 3200 MHz, and an AMD Ryzen 9 5900X 12-core CPU operating at a base frequency of 3.7 GHz. The host operating system was Ubuntu 22.04.3 LTS with CUDA 12.1 and cuDNN 8.9. The Python runtime was version 3.11.4 managed via conda. Key Python dependencies included LangChain 0.1.16, FastAPI 0.110.0, psycpg2 2.9.9, PyMuPDF 1.23.26, and Diffusers 0.27.0. The frontend was implemented in React.js 18.2.0 with TypeScript 5.3.

B. Document Ingestion and Index Construction

The full 87,342-chunk corpus was ingested over approximately 4.2 hours, with the majority of elapsed time attributable to OpenAI API round-trip latency for embedding generation (mean 38 ms/chunk, processed in batches of 100 with exponential backoff retry logic). Following ingestion, HNSW index construction on the embedding column completed in 23 minutes. Index size on disk was 2.1 GB for the 87,342-chunk corpus at float32 vector precision. Total database size including text content and metadata was 4.8 GB.

C. Baseline Configuration

The retrieval-free baseline was implemented as the identical system architecture with the retrieval component disabled: user queries were submitted directly to GPT-4o without any retrieved context, using the same system prompt (minus the context-grounding instruction) and generation parameters. This configuration isolates the contribution of the RAG retrieval component to observed performance differences, ensuring that observed gains are attributable to retrieval augmentation rather than model-level or prompt-level factors.

D. HNSW Index Parameter Sensitivity Analysis

To characterise the sensitivity of retrieval performance to HNSW index parameters, a systematic grid search was conducted over $m \in \{8, 16, 32\}$ and $ef_construction \in \{32, 64, 128\}$ on a held-out validation set of 100 queries. Recall@5 peaked at 91.8% for $m=32$, $ef_construction=128$, at the cost of a $3.8\times$ increase in index build time relative to the selected configuration ($m=16$, $ef_construction=64$, Recall@5=91.4%). The marginal recall improvement of 0.4 percentage points was judged insufficient to justify the substantially higher build-time cost for the target corpus size, motivating the selected parameter configuration.

VI. RESULTS AND DISCUSSION

A. Quantitative Performance Analysis

Table III presents the comprehensive quantitative evaluation results across all system configurations and the retrieval-free baseline. The RAG-augmented system configurations consistently and substantially outperform the baseline across every applicable evaluation metric.

TABLE III Comprehensive Quantitative Evaluation Results

Metric	GPT-4o+RAG	LLaMA-3+RAG	DALL-E 3	Baseline
Recall@5 (%)	91.4	88.7	N/A	74.2
Faithfulness (RAGAS)	0.89	0.84	N/A	0.61
Answer Relevance	0.91	0.87	N/A	0.68
Context Precision	0.86	0.81	N/A	N/A
BLEU-4 Score	0.74	0.69	N/A	0.48
FID Score	N/A	N/A	12.3	N/A
Latency (ms)	1,240	980	3,100	420
User Satisfaction	4.6/5	4.3/5	4.5/5	3.1/5
Hallucination (%)	6	11	N/A	34

GPT-4o + RAG achieves the highest overall performance across all text generation metrics. The Recall@5 of 91.4% represents a 23.2 percentage point improvement over the retrieval-free baseline (74.2%), reflecting both the quality of the text-embedding-3-small embedding model and the effectiveness of the HNSW index configuration. The RAGAS answer faithfulness score of 0.89 versus 0.61 for the baseline (a 46% relative improvement) quantifies the degree to which the RAG pipeline grounds generated responses in verifiable retrieved content. The BLEU-4 score of 0.74 (vs. 0.48 baseline) confirms strong lexical alignment with expert-authored reference responses. Critically, hallucination rates are reduced from 34% in the baseline to 6% in the RAG + GPT-4o configuration—an 82% relative reduction—representing the most practically significant finding of this evaluation.

LLaMA-3 + RAG delivers highly competitive performance, achieving Recall@5 of 88.7%, faithfulness of

0.84, and user satisfaction of 4.3/5, while providing a 21% reduction in response latency (980 ms vs. 1,240 ms for GPT-4o) and eliminating per-token API inference costs entirely. The marginal performance gap between LLaMA-3 and GPT-4o across all metrics demonstrates that open-source models, when augmented with high-quality retrieval infrastructure, can closely approximate the performance of proprietary counterparts at substantially lower operational cost. This finding has significant implications for enterprise deployments requiring data sovereignty, cost predictability, or offline operation.

The increase in end-to-end latency relative to the baseline (420 ms) reflects the overhead of embedding generation (≈ 45 ms), HNSW search (≈ 12 ms), and prompt assembly (≈ 5 ms), with LLM response time constituting the dominant contribution. This latency overhead is judged well within acceptable bounds for conversational applications given the commensurate and substantial gains in response quality and factual accuracy.

B. RAGAS Metric Decomposition

Beyond the aggregate faithfulness score, the RAGAS framework provides decomposed metrics that illuminate specific aspects of RAG system behaviour. The answer relevance score—measuring the degree to which the generated response addresses the original query intent—stands at 0.91 for GPT-4o + RAG and 0.87 for LLaMA-3 + RAG, reflecting the effectiveness of the augmented prompt construction in maintaining query focus. The context precision score—measuring the proportion of retrieved context that is actually utilised in the response—reaches 0.86 for GPT-4o + RAG, indicating that the system efficiently leverages retrieved information without over-reliance on irrelevant passages. These decomposed metrics collectively confirm that the RAG pipeline achieves high retrieval quality, effective context utilisation, and strong response relevance simultaneously.

C. Qualitative Analysis of Generated Responses

Qualitative inspection of 50 randomly selected query-response pairs from each configuration revealed systematic and consistent differences in response characteristics. The baseline GPT-4o configuration frequently produced confident, syntactically polished responses containing specific technical claims—exact parameter values, algorithm names, performance metrics—that could not be verified against any source document and were in several cases demonstrably incorrect. This pattern was particularly pronounced for domain-specific technical queries where training data coverage was sparse or outdated.

In contrast, the RAG + GPT-4o configuration consistently anchored its responses in retrieved document passages, citing specific sources and in several cases explicitly acknowledging the limits of available retrieved information for queries lying at the periphery of corpus coverage. Evaluators noted that RAG responses were not only more factually accurate but also more precisely calibrated to query specificity: broad conceptual queries received appropriately general responses drawing on multiple retrieved passages, while narrow technical queries received precise, source-attributed answers.

A recurring failure mode in the LLaMA-3 configuration was observed for complex multi-hop queries requiring the synthesis of information from three or more retrieved chunks: LLaMA-3 occasionally prioritised the highest-similarity chunk and underweighted supplementary context, producing partially complete responses. GPT-4o demonstrated superior multi-document synthesis, attributable to its larger effective context window and more advanced cross-document reasoning capabilities.

D. Image Generation Quality Assessment

User evaluations of image generation quality across 50 test prompts yielded mean relevance and visual quality scores of 4.5/5 and 4.3/5 respectively for DALL-E 3, and 4.1/5 and 3.9/5 for SDXL. The FID score of 12.3 for DALL-E 3 (computed against a reference set of 500 real domain-relevant images) indicates high perceptual quality and distributional fidelity. SDXL produced visually comparable outputs for single-concept prompts but exhibited measurably degraded spatial coherence and semantic fidelity for compositionally complex prompts—for example, those specifying multiple objects with precise spatial relationships or requiring adherence to specific technical diagram conventions. This limitation is consistent with known challenges in compositional generation for denoising diffusion models [7].

A key finding is that RAG-enriched image generation prompts consistently produced more technically accurate and domain-appropriate images than those derived from raw user queries alone. For example, a query requesting "an illustration of the HNSW graph index structure" generated with RAG-retrieved context describing the multi-layer graph construction algorithm produced a geometrically accurate hierarchical graph diagram, whereas the same prompt without retrieved context produced a generic network diagram lacking HNSW-specific structural properties. This finding demonstrates that the integration of RAG into the image generation pipeline provides meaningful quality improvements beyond the text generation modality.

E. Retrieval Latency and Scalability Analysis

To characterise retrieval scalability, ANN search latency was measured across corpus sizes ranging from 10,000 to 87,342 chunks. Mean query latency increased from 4.2 ms at 10,000 chunks to 12.8 ms at 87,342 chunks, consistent with the sub-linear $O(\log n)$ asymptotic complexity of HNSW search. Extrapolating this growth curve, retrieval latency is projected to reach approximately 35 ms at 200,000 chunks and 85 ms at 500,000 chunks—both within acceptable bounds for conversational applications. Beyond approximately 1 million chunks, the IVF-PQ indexing strategy available in pgvector is recommended to maintain sub-100 ms retrieval latency, at the cost of a marginal reduction in recall.

F. Ablation Study

To isolate the contribution of individual system components, an ablation study was conducted across four configurations: (i) No RAG (baseline); (ii) RAG with BM25 sparse retrieval (replacing HNSW dense retrieval); (iii) RAG with dense retrieval but no chunk overlap; and (iv) the full system (dense retrieval + 50-token overlap). Configurations

(ii) and (iii) achieved intermediate Recall@5 values of 81.3% and 88.6% respectively, confirming that both dense retrieval over sparse alternatives and chunk overlap over non-overlapping segmentation make meaningful independent contributions to overall retrieval quality.

G. Limitations

Several limitations of the current implementation are acknowledged. First, the offline HNSW index construction does not support incremental updates: corpus additions require full index rebuilds, which at 87,342 chunks require approximately 23 minutes—an operationally costly procedure for dynamically updated knowledge bases. Second, the intent classification module for modality routing (text vs. image) relies on keyword heuristics that produce routing errors for 7% of ambiguous test queries; a learned binary classifier trained on historical query logs would offer substantially improved robustness. Third, the evaluation corpus, while carefully constructed, may not fully represent the distribution of real-world user queries in production deployments, potentially introducing evaluation bias. Fourth, SDXL image generation latency (8–12 seconds on the local GPU) may unacceptably degrade user experience in latency-sensitive interactive applications, motivating investigation of faster sampling schedulers such as LCM [Luo et al., 2023].

VII. CONCLUSION

This paper has presented the comprehensive design, implementation, and empirical evaluation of a multimodal Retrieval-Augmented Generation AI assistant integrating dense semantic retrieval, dual language model generation, and text-to-image synthesis within a unified, modular microservice architecture. The system was evaluated against a retrieval-free baseline using a comprehensive suite of quantitative and qualitative metrics, demonstrating substantial and consistent improvements across all dimensions of assessment.

The most significant empirical finding is the 82% relative reduction in hallucination rates achieved by the RAG + GPT-4o configuration relative to the baseline (6% vs. 34%), accomplished without any modification to the underlying language model's parameters. This result unambiguously confirms that retrieval augmentation constitutes an effective, training-free mechanism for grounding conversational AI in verifiable factual knowledge—a finding with substantial implications for the safe deployment of AI systems in knowledge-sensitive domains. The comparative evaluation further demonstrates that open-source models such as LLaMA-3, when augmented with strong retrieval infrastructure, can closely approximate the performance of proprietary counterparts while offering meaningful advantages in operational cost, data privacy, and deployment flexibility.

The integration of image generation into the RAG pipeline, and the demonstrated quality improvement of RAG-enriched image generation prompts over raw-query prompts, establishes a proof-of-concept for genuinely multimodal knowledge-augmented conversational AI. The system achieves high user satisfaction (4.6/5 for text, 4.5/5 for image

generation) and demonstrates practical scalability to corpus sizes exceeding 87,000 document chunks with sub-15 ms retrieval latency.

VIII. FUTURE SCOPE

The findings and limitations of the present work motivate several promising directions for future research and system development:

- Domain-specific fine-tuning of LLaMA-3 via parameter-efficient methods such as Low-Rank Adaptation (LoRA) and Quantised LoRA (QLoRA) on the target knowledge corpus is expected to narrow the performance gap with GPT-4o on complex technical queries while preserving the cost and privacy advantages of local deployment. Preliminary experiments with LoRA fine-tuning on a 10,000-sample subset of the corpus suggest a projected faithfulness improvement of 0.04–0.06 RAGAS points.
- Agentic RAG extensions incorporating multi-step reasoning and iterative retrieval loops—inspired by Self-RAG [14] and ReAct [21]—would enable the system to decompose complex multi-hop questions into constituent sub-queries, retrieve targeted information for each sub-query, and synthesise a comprehensive final response. This capability would directly address the multi-hop reasoning limitations observed in the LLaMA-3 configuration.
- Semantic caching of high-frequency query-embedding pairs using an LRU cache would eliminate redundant embedding API calls and reduce end-to-end latency for repeated or near-duplicate queries in production deployments with realistic query distributions.
- Incremental HNSW index maintenance, anticipated in forthcoming pgvector releases, would enable real-time corpus expansion without full index rebuilds, transforming the system into a continuously learning knowledge assistant capable of incorporating newly published research and technical documentation without service interruption.
- Extension of multimodal outputs to include video generation (via models such as Sora or RunwayML Gen-3) and speech synthesis (via ElevenLabs or Bark) would substantially broaden the system's applicability to educational content creation, accessibility tools for visually or hearing-impaired users, and interactive multimedia knowledge exploration.
- Cloud-native deployment on a managed Kubernetes platform with GPU node pools, horizontal pod autoscaling for the LLM serving layer, and a distributed pgvector cluster would enable production-grade scalability to support thousands of concurrent users, with automated failover and load balancing ensuring high availability.
- Investigation of privacy-preserving RAG variants, including federated retrieval over distributed private knowledge bases and differential privacy mechanisms for embedding generation, would extend the system's applicability to healthcare, legal, and financial domains with stringent data governance requirements.

ACKNOWLEDGMENT

The authors sincerely thank Supervisor Saurabh Pathak for continuous mentorship, valuable insights, and constructive guidance throughout all phases of this project. Appreciation is also extended to the open-source communities behind LangChain, Ollama, pgvector, and the Hugging Face Diffusers library, whose foundational tools enabled the successful implementation of this work. The authors also acknowledge the evaluator panel for their time and diligence in conducting the human evaluation study.

REFERENCES

- [1] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
- [2] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv:2307.09288, 2023.
- [3] R. Anil et al., "Gemini: A Family of Highly Capable Multimodal Models," arXiv:2312.11805, 2023.
- [4] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [5] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [6] A. Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents," arXiv:2204.06125, 2022.
- [7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in *Proc. CVPR*, pp. 10684–10695, 2022.
- [8] pgvector, "Open-source vector similarity search for Postgres," 2024. [Online]. Available: <https://github.com/pgvector/pgvector>
- [9] A. Vaswani et al., "Attention Is All You Need," in *Proc. NeurIPS*, vol. 30, 2017.
- [10] Meta AI, "Introducing Meta Llama 3," 2024. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
- [11] T. Brown et al., "Language Models are Few-Shot Learners," in *Proc. NeurIPS*, vol. 33, pp. 1877–1901, 2020.
- [12] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in *Proc. ICML*, 2020.
- [13] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in *Proc. EACL*, 2021.
- [14] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv:2310.11511, 2023.
- [15] Z. Jiang, F. F. Xu, L. Gao et al., "Active Retrieval Augmented Generation," in *Proc. EMNLP*, 2023.
- [16] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020.
- [17] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbour Search Using HNSW Graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2018.
- [18] A. Ramesh et al., "Zero-Shot Text-to-Image Generation," in *Proc. ICML*, 2021.
- [19] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in *Proc. ICML*, 2021.
- [20] S. Es, J. James, L. E. Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv:2309.15217, 2023.
- [21] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.
- [22] N. Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," arXiv:2210.07316, 2022.

- [23]E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in Proc. ICLR, 2022.
- [24]T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," in Proc. NeurIPS, 2023.
- [25]K. Papineni, S. Roukos, T. Ward, and W. J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proc. ACL, pp. 311–318, 2002.





HRIT UNIVERSITY

ISO 9001:2015 & ISO 14001:2015

Formerly HRIT GROUP OF INSTITUTIONS, Since 2005

(Established under the provision of the Uttar Pradesh Private Universities Act 2019)

NATIONAL CONFERENCE (NGAISL-2026)

on

Next GEN Artificial Intelligence From Smart Learning to Smart Living

April 4th, 2026

CERTIFICATE OF ACHIVEMENT

This is to certify that Dr./Mr./Ms. Abhishek Prajapati of
Computer Science & Engineering Participated/presented
Paper titled Multi model RAG-Based AI Assistant
in the National Conference on "Next GEN Artificial Intelligence From smart learning to Smart
Living Faculty of Computer Application and Faculty of Computer Engineering & Technology
on April 4th 2026

Prof. (Dr.) Anubhav Kumar
Dean Academics
Conference Chair

Prof. (Dr.) Naveen Kumar Singh
Dean FOCA
Conference Convenor

Prof. (Dr.) UPMA Sharma
Dean FOET
Conference Convenor



HRIT UNIVERSITY

ISO 9001:2015 & ISO 14001:2015

Formerly HRIT GROUP OF INSTITUTIONS, Since 2005

(Established under the provision of the Uttar Pradesh Private Universities Act 2019)

NATIONAL CONFERENCE (NGAISL-2026)

on

Next GEN Artificial Intelligence From Smart Learning to Smart Living

April 4th, 2026

CERTIFICATE OF ACHIVEMENT

This is to certify that Dr./Mr./Ms. Atul Kumar of
Computer Science & Engineering Participated/presented
Paper titled Multi model RAG-Based AI Assistant
in the National Conference on "Next GEN Artificial Intelligence From smart learning to Smart
Living Faculty of Computer Application and Faculty of Computer Engineering & Technology
on April 4th 2026

Prof. (Dr.) Anubhav Kumar
Dean Academics
Conference Chair

Prof. (Dr.) Naveen Kumar Singh
Dean FOCA
Conference Convenor

Prof. (Dr.) UPMA Sharma
Dean FOET
Conference Convenor



— END OF PROJECT REPORT —

Department of Computer Science and Engineering
NITRA Technical Campus, Ghaziabad – 201002, Uttar Pradesh, India
Session: 2025 – 2026